Contents

1 Verilog HDL

Reference basis: "Verilog HDL by Samir Palnitkar" - written/compiled by Maitreya Ranade.

- Overview of Digital Design with Verilog HDL
- Hierarchical Modeling Concepts
- Basic Concepts
- Modules & Ports
- Gate-Level Modeling
- Dataflow Modeling
- Behavioral Modeling
- Tasks & Functions
- Useful Modeling Techniques
- Logic Synthesis With Verilog
- Advanced Verification Techniques

---

2 Extra notes

- IEEE 1364-2005 Verilog HDL
- IEEE 1800-2023 SystemVerilog
- HDL Comparison

3 Overview of Digital Design with Verilog HDL

3.1 1. Evolution of Digital Design and EDA

Digital circuit design has evolved significantly from the use of vacuum tubes to transistors and eventually to integrated circuits (ICs). Early ICs were categorized based on scale:

- Small Scale Integration (SSI) contained a very small number of gates.
- Medium Scale Integration (MSI) increased capacity to hundreds of gates.
- Large Scale Integration (LSI) enabled thousands of gates per chip.
- Very Large Scale Integration (VLSI) allowed more than 100,000 transistors on a single chip.

As integration levels increased, manual design methods became impractical due to complexity. This led to the development of **Electronic Design Automation (EDA)** tools.

EDA combines: - CAD (Computer-Aided Design): Physical design tasks such as layout, placement, and routing. - CAE (Computer-Aided Engineering): Front-end processes such as simulation, synthesis, and timing analysis.

With VLSI, physical prototyping using breadboards became infeasible. Designers increasingly relied on simulation tools to verify functionality before fabrication. Logic simulation became a critical step to identify and fix functional issues early in the design cycle.

3.2 2. Emergence of Hardware Description Languages (HDLs)

Traditional programming languages such as C, Pascal, and FORTRAN are sequential in nature and unsuitable for modeling hardware, which operates concurrently.

Hardware Description Languages (HDLs) were introduced to model parallelism in digital systems. The most prominent HDLs are:

- Verilog HDL (introduced in 1983 by Gateway Design Automation)
- VHDL (developed under DARPA funding)

Initially, HDLs were used mainly for simulation and verification. Designers still had to manually convert HDL descriptions into gate-level schematics.

The introduction of logic synthesis in the late 1980s transformed this workflow. Designers could describe circuits at the Register Transfer Level (RTL), specifying data flow and behavior. Logic synthesis tools automatically translated RTL descriptions into gate-level implementations.

This shift enabled designers to focus on functionality rather than low-level gate connections. HDLs also became widely used for system-level design, including modeling of FPGAs, buses, and complete systems.

Verilog HDL was standardized as IEEE 1364-1995 and later updated in 2001.

3.3 3. Typical VLSI Design Flow

The modern VLSI design flow using HDLs consists of the following stages:

1. Specification
   The design process begins with defining the functional requirements, architecture, and interfaces of the system. At this stage, implementation details are not considered.

2. Behavioral Description
   A high-level model of the system is created to evaluate functionality, performance, and compliance. This description is often written using HDLs.

3. RTL Design
   The behavioral model is refined into a Register Transfer Level (RTL) description. This stage defines how data moves between registers and how operations are performed.

4. Logic Synthesis
   RTL descriptions are converted into a gate-level netlist. The synthesis process ensures that the design meets constraints related to timing, area, and power.

5. Place and Route
   The gate-level netlist is mapped onto a physical layout. This includes placement of components and routing of interconnections.

6. Verification and Fabrication
   The physical design is verified before being sent for fabrication.

Most design effort is concentrated at the RTL level, where optimization has the greatest impact. RTL-based design significantly reduces development time from years to months and allows multiple design iterations.

Behavioral synthesis tools, which convert high-level descriptions directly into RTL, are emerging but not yet widely adopted.

It is important to note that EDA tools are only as effective as their inputs. Poorly written RTL can result in inefficient designs, following the principle of "Garbage In, Garbage Out (GIGO)".

3.4 4. Importance of HDLs

HDLs provide several advantages over traditional schematic-based design:

- Technology Independence
  Designers can write RTL without targeting a specific fabrication technology. The same design can be synthesized for different technologies with appropriate optimization.

- Early Functional Verification
  Bugs can be identified and resolved at the RTL stage, reducing the likelihood of costly errors in later stages.

- Higher Level of Abstraction
  Designers can focus on system behavior and data flow rather than low-level gate implementation.

- Improved Readability and Maintainability
  Text-based descriptions with comments are easier to understand and debug compared to complex schematics.

HDLs are now the standard approach for designing complex digital systems and are indispensable for modern digital designers.

3.5 5. Popularity of Verilog HDL

Verilog HDL has become one of the most widely used hardware description languages due to the following features:

- It has a syntax similar to the C programming language, making it easy to learn for software engineers.
- It supports multiple levels of abstraction, including behavioral, RTL, gate-level, and switch-level modeling.
- It is supported by most logic synthesis and simulation tools.
- It is widely supported by semiconductor vendors through standard libraries.
- It includes the Programming Language Interface (PLI), which allows integration with C code for customization and advanced functionality.

These advantages have made Verilog a preferred choice for both design and verification.

3.6 6. Trends in HDLs and Digital Design

3.6.1 Increasing Abstraction

As circuit complexity continues to grow, designers are working at higher levels of abstraction. They focus on functionality, while EDA tools handle implementation details.

3.6.2 Dominance of RTL Design

RTL-based design remains the most widely used methodology. Logic synthesis tools efficiently convert RTL into gate-level netlists.

Behavioral synthesis, which operates at a higher level of abstraction, exists but has not achieved widespread adoption.

3.6.3 Advances in Verification

New verification techniques have emerged to improve design correctness:

- Formal Verification
  Uses mathematical methods to prove correctness and equivalence between RTL and gate-level designs.
- Assertion-Based Verification
  Embeds checks within RTL code to validate critical conditions during simulation.
- Advanced Verification Languages
  Combine hardware modeling with object-oriented programming concepts to enable automated testing and coverage analysis. These languages complement, rather than replace, HDLs.

3.6.4 High-Performance Design Considerations

For timing-critical designs such as microprocessors, purely RTL-based synthesis may not yield optimal results. Designers often incorporate gate-level elements into RTL descriptions to achieve better timing performance.

3.6.5 System-Level Design Methodology

A mixed bottom-up approach is commonly used in large systems. Designers combine:

- Custom RTL modules
- Behavioral models
- Vendor-provided IP cores

This approach enables faster system integration, early simulation, and reduced development costs.

---

4 Hierarchical Modeling Concepts

- Hierarchical modeling is a fundamental concept in digital design that enables designers to manage complexity by structuring designs into smaller, reusable components.
- A well-defined design methodology is essential for efficient HDL-based design.
- A digital system is composed of multiple interconnected components. Understanding how these components are structured and interact is critical for both design and simulation.

4.1 1. Design Methodologies

Digital design primarily follows two methodologies: top-down and bottom-up. In practice, a combination of both is used.

4.1.1 Top-Down Design Methodology

- The design process begins by defining the top-level system functionality.
- The system is progressively decomposed into smaller sub-blocks.
- This decomposition continues until indivisible units, called leaf cells, are reached.
- This approach emphasizes system-level planning and architecture.

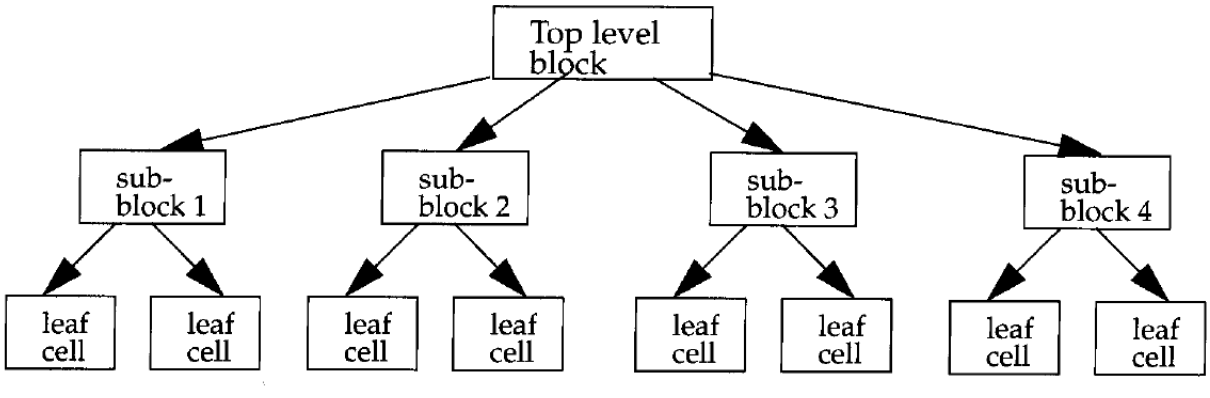


Figure 1: Top-Down Design Methodology

4.1.2 Bottom-Up Design Methodology

- The design begins with basic building blocks such as logic gates or primitive cells.
- These blocks are combined to form higher-level modules.
- The process continues until the complete system is constructed.
- This approach focuses on optimized implementation of low-level components.

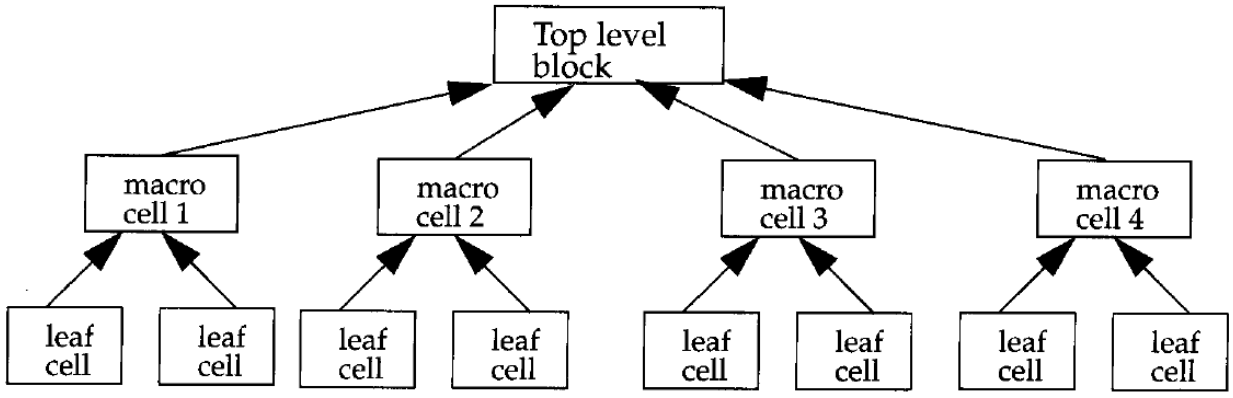


Figure 2: Bottom-Up Design Methodology

4.1.3 Combined Approach

- In real-world designs, both methodologies are used together.
- System architects define the overall structure using a top-down approach.
- Circuit designers simultaneously develop optimized leaf cells using a bottom-up approach.
- The two approaches converge at an intermediate level, typically at standard building blocks such as flip-flops.

4.2 2. Modules in Verilog

A module is the fundamental building block in Verilog.

- A module can represent either a simple element or a complex system composed of submodules.
- It provides functionality through a defined interface consisting of input and output ports.
- The internal implementation of a module is hidden from other modules, enabling abstraction and modularity.
- This encapsulation allows internal changes without affecting the rest of the design.

A module is defined using the module and endmodule keywords and includes: - A unique module name - A list of input and output ports - Internal implementation

---

4.3 3. Levels of Abstraction in Verilog

Verilog supports multiple levels of abstraction to describe the same hardware module. The external behavior of the module remains consistent regardless of abstraction level.

4.3.1 Behavioral Level

- This is the highest level of abstraction.
- The design is described using algorithms without specifying hardware details.
- It is similar to high-level programming languages like C.

4.3.2 Dataflow Level

- The design is expressed in terms of data movement between registers.
- The designer specifies how data is processed and transferred.

4.3.3 Gate Level

- The design is described using logic gates and their interconnections.
- This corresponds closely to schematic-based design.

4.3.4 Switch Level

- This is the lowest level of abstraction.
- The design is described using switches and storage nodes.
- It requires detailed knowledge of transistor-level behavior.

4.3.5 Key Observations

- Designers can mix different abstraction levels within the same design.
- RTL (Register Transfer Level) typically combines behavioral and dataflow modeling.
- Higher abstraction levels provide better flexibility and portability.
- Lower abstraction levels offer more control but reduce flexibility and increase complexity.

4.4 4. Module Instantiation and Instances

A module acts as a template from which instances are created.

- Instantiation is the process of creating a specific instance of a module.
- Each instance is a unique object with its own name, signals, and parameters.
- Multiple instances of the same module can exist in a design.

For example, a ripple carry counter may instantiate multiple T flip-flop modules, each with unique connections.

4.4.1 Important Rules

- Each instance must have a unique name.
- Module definitions cannot be nested inside other module definitions.

- Modules must be instantiated to be used; defining a module alone does not include it in the design.

This distinction between module definition and module instance is critical in Verilog design.

4.5 7. Components of a Simulation

After designing a module, its functionality must be verified through simulation.

4.5.1 Design Block

- Represents the actual hardware design to be tested.

4.5.2 Stimulus Block (Testbench)

- Provides input signals to the design block.
- Observes and verifies output responses.
- Typically written in Verilog.

It is considered good practice to keep the design and stimulus blocks separate to improve modularity and reusability.

4.5.3 Methods of Applying Stimulus

4.5.3.1 Method 1: Stimulus as Top-Level Module

- The stimulus block instantiates the design block.
- It directly drives input signals and monitors outputs.
- The stimulus block acts as the top-level module.

4.5.3.2 Method 2: Separate Top-Level Module

- A dummy top-level module instantiates both the design and stimulus blocks.
- The stimulus interacts with the design through defined interfaces.
- This approach improves modularity and separation of concerns.

Both methods are valid and can be chosen based on design and testing requirements.

---

5 Verilog Basic Concepts

5.1 1. Overview

This chapter establishes the foundational constructs and conventions used in Verilog HDL. These concepts are essential for understanding how Verilog models real hardware behavior, including data representation, simulation, and coding structure.

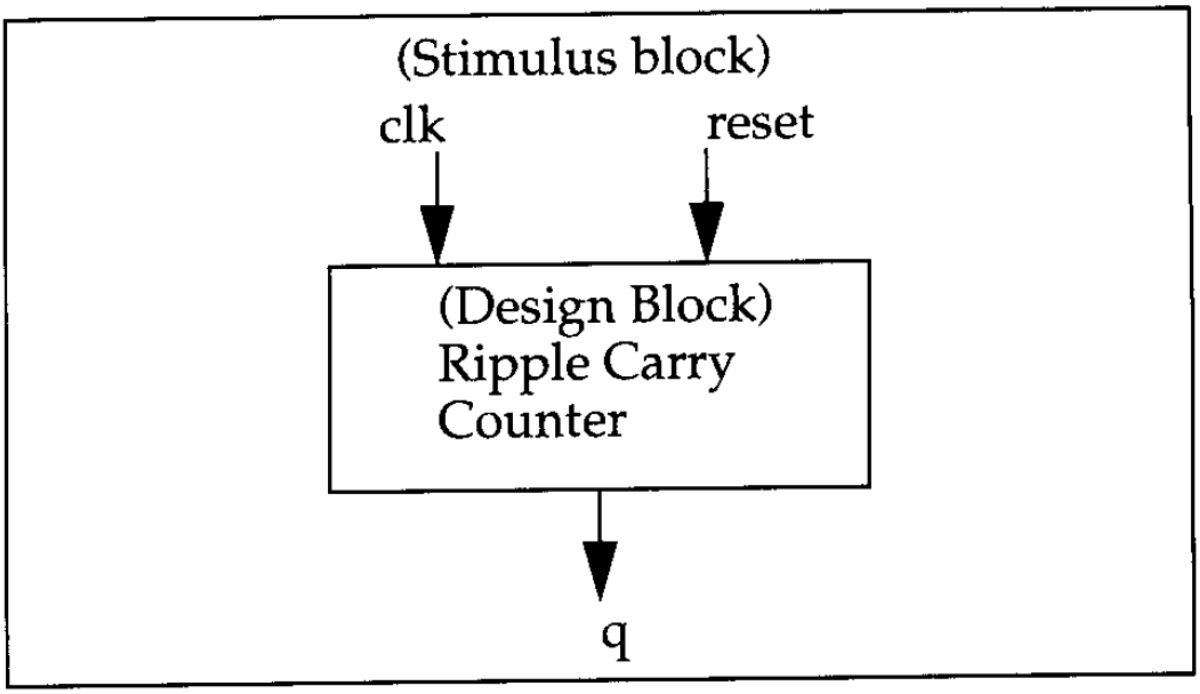


Figure 3: Stimulus as Top-Level Module

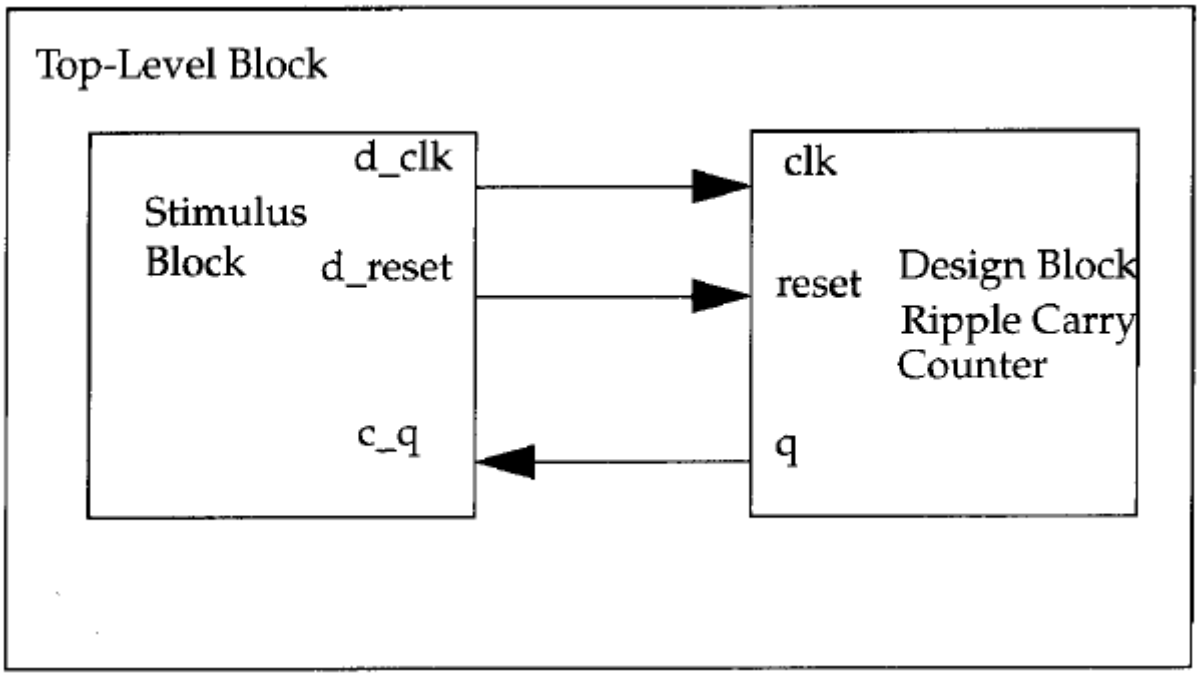


Figure 4: Separate Top-Level Module

5.2 2. Lexical Conventions

Verilog syntax is similar to the C programming language and consists of tokens such as identifiers, keywords, numbers, operators, and strings.

5.2.1 General Rules

- Verilog is a case-sensitive language.
- Keywords are always written in lowercase.
- Code is interpreted as a stream of tokens.

5.2.2 Whitespace

- Spaces, tabs, and newlines are ignored except when separating tokens.
- Whitespace inside strings is preserved.

5.2.3 Comments

- Single-line comments start with //.
- Multi-line comments are enclosed within /* */.
- Nested multi-line comments are not allowed.

5.2.4 Operators

- Unary operators act on a single operand.
- Binary operators operate between two operands.
- Ternary operators use the conditional format ?:.

5.2.5 Number Representation

- Numbers can be specified as sized or unsized.

5.2.5.1 Sized Numbers

- Format: <size>'<base><value>
- Supported bases: binary (b), decimal (d), hexadecimal (h), octal (o)
- Eg. 4'b1111; 12'habc;

5.2.5.2 Unsized Numbers

- Default to decimal and at least 32 bits.
- Eg. 'b1111; 'hc3; // Both of these are 32 bit numbers

5.2.5.3 Special Values

- x represents unknown values.
- z represents high impedance (floating state).
- ? can be used as an alternative to z in specific contexts.

5.2.5.4 Additional Rules

- Negative numbers use a minus sign before the size. Eg. -6'd3;
- Underscores improve readability and are ignored. Eg. 8'b1110_0111;
- Automatic bit extension applies based on the most significant bit.

5.2.6 Strings

- Strings are enclosed in double quotes.
- They must be contained on a single line.
- Each character occupies one byte (ASCII).

5.2.7 Identifiers and Keywords

- Identifiers name variables, modules, and signals.
- They consist of letters, digits, underscores, and $.
- They must not start with a digit or $.

5.2.8 Escaped Identifiers

- Begin with \ and end with whitespace.
- Allow use of special characters in names.

5.3 3. Data Types in Verilog

Verilog data types closely model hardware behavior and signal representation.

5.3.1 Value Set

Verilog supports four logic values: - 0 represents logic low. - 1 represents logic high. - x represents unknown. - z represents high impedance.

Signal strengths are also defined to resolve conflicts between multiple drivers.

5.4 4. Nets

- Nets represent physical connections between hardware elements.
- They continuously reflect the value driven by connected devices.
- The most commonly used net type is wire.
- Default value of a net is z if undriven.

Key characteristics: - Nets do not store values. - They must be driven by some source.

5.5 5. Registers

- Registers represent storage elements in Verilog.
- They retain their value until explicitly changed.

- Declared using the reg keyword.

Important distinctions: - A Verilog register is not necessarily a physical flip-flop. - Registers do not require a clock to update values. - Default value of a register is x.

Registers can also be declared as signed for arithmetic operations.

5.6 6. Vectors

- Vectors represent multi-bit nets or registers.
- Declared using range notation such as [msb:lsb].

Key concepts: - Default is scalar (1-bit) if no range is specified. - Bit ordering determines significance, with the left index as MSB.

5.6.1 Vector Operations

- Individual bits and ranges can be accessed using indexing.
- Part-select allows selection of subsets of bits.
- Variable part-select enables dynamic access using expressions.

5.7 7. Integer, Real, and Time Data Types

5.7.1 Integer

- Used for general-purpose computations.
- Typically at least 32 bits and signed.

5.7.2 Real

- Used for floating-point values.
- Supports decimal and scientific notation.
- Values are rounded when assigned to integers.

5.7.3 Time

- Used to store simulation time.
- Typically at least 64 bits.
- Retrieved using the system function $time.

5.8 8. Arrays

- Arrays allow storage of multiple elements of the same type.
- Can be one-dimensional or multi-dimensional.

Key characteristics: - Each element can be a scalar or vector. - Accessed using index notation. - Arrays differ from vectors, which represent a single multi-bit value.

5.9 9. Memories

- Memories are modeled as one-dimensional arrays of registers.
- Used to represent RAM, ROM, and register files.

Key points: - Each element represents a word. - Each word can be multiple bits wide. - Access is performed using a single index.

5.10 10. Parameters and Constants

- Parameters define constants within a module.
- Declared using the parameter keyword.

Key advantages: - Enable configurable and reusable designs. - Can be overridden during module instantiation.

5.10.1 Local Parameters

- Declared using localparam.
- Cannot be modified externally.
- Useful for defining fixed internal constants such as state encodings.

5.11 11. Strings in Registers

- Strings can be stored in register variables.
- Each character occupies 8 bits.
- If the register is wider than the string, unused bits are zero-filled.
- If narrower, the string is truncated.

Special characters such as newline and tab are supported using escape sequences.

5.12 12. System Tasks

System tasks are built-in functions used for simulation control and debugging. They are identified by a $ prefix.

5.12.1 Displaying Information

- $display prints values, variables, or expressions.
- Supports formatted output similar to C's printf.
- Automatically appends a newline.

5.12.2 Monitoring Signals

- $monitor continuously tracks specified signals.
- Outputs values whenever any monitored signal changes.
- Only one active monitor is allowed at a time.

5.12.3 Simulation Control

- $stop pauses simulation for debugging.
- $finish terminates simulation execution.

5.13 13. Compiler Directives

Compiler directives control preprocessing behavior and are identified by a backtick (‘).

5.13.1 `define`

- Used to create macros or constants.
- Improves readability and maintainability.

5.13.2 `include`

- Used to include contents of another file.
- Commonly used for header files and shared definitions.

---

6 Verilog Modules and Ports

6.1 1. Overview of Modules

A module is the fundamental building block in Verilog and represents a self-contained unit of functionality. A module encapsulates internal implementation while exposing an interface through ports.

6.1.1 Structure of a Module

A Verilog module typically consists of the following components:

- Module declaration, which includes the keyword module, module name, and optional port list.
- Port declarations and optional parameters defined at the beginning.
- Internal components, which may include:
  - Variable declarations
  - Dataflow statements (assign)
  - Behavioral blocks (initial , always)
  - Instantiations of lower-level modules
  - Tasks and functions
- The module definition always ends with the keyword endmodule.

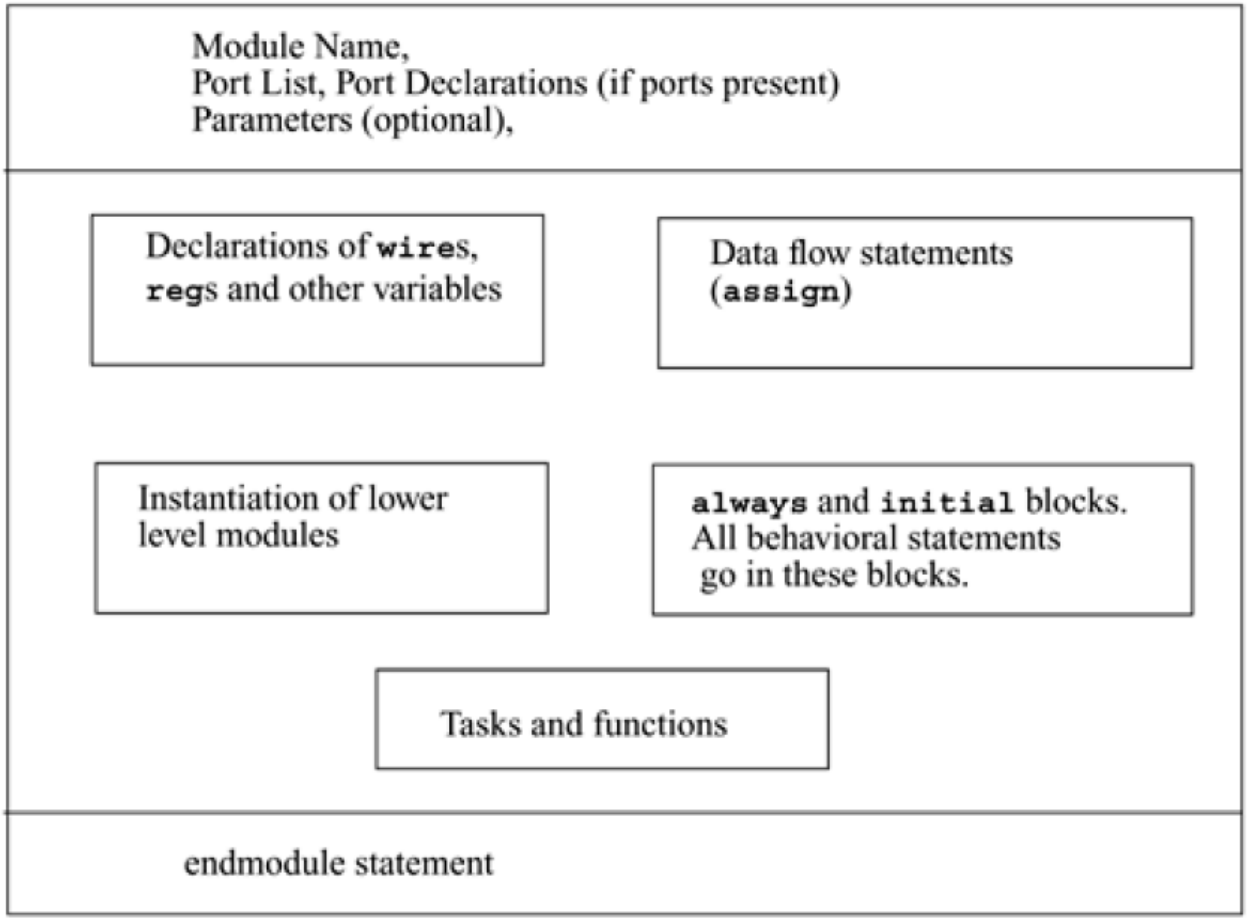


Figure 5: Structure of a verilog module

Only the module declaration and endmodule are mandatory. All other components are optional and can be arranged in any order within the module.

Verilog allows multiple modules to be defined within a single file, and their order of definition is not constrained.

6.2 2. Ports and Module Interface

Ports define the interface through which a module communicates with its external environment. They are analogous to input/output pins of a hardware component.

6.2.1 Key Characteristics

- Ports are the only visible interface of a module.
- Internal implementation is hidden, enabling abstraction and modular design.
- Changes to internal logic do not affect external connections as long as the port interface remains unchanged.

6.3 3. Port List

- A module may optionally include a list of ports.
- If a module does not interact with external signals, it may have no port list.
- A top-level module in simulation often has no ports because it acts as the root.

Example: - A functional module such as an adder includes ports. - A testbench or top-level module may not include ports.

6.4 4. Port Declarations

Each port in the port list must be declared with its direction:

- input defines incoming signals.
- output defines outgoing signals.
- inout defines bidirectional signals.

6.4.1 Important Rules

- Ports are implicitly of type wire unless specified otherwise.
- Input and inout ports must always be of type net (wire).
- Output ports can be of type wire or reg.

If an output needs to store its value across time (for example, in sequential logic), it must be declared as reg.

6.4.2 ANSI C Style Declaration

Verilog supports an alternative syntax where port direction and type are declared directly in the module header. This avoids duplication and improves readability.

6.5 5. Port Connection Rules

When connecting modules during instantiation, specific rules must be followed:

6.5.1 Inputs

- Internally, inputs must be nets.
- Externally, inputs can connect to either nets or registers.

6.5.2 Outputs

- Internally, outputs can be nets or registers.
- Externally, outputs must connect to nets and cannot connect directly to registers.

6.5.3 Inouts

- Internally and externally, inout ports must always be nets.

6.5.4 Width Matching

- Connections between ports of different widths are allowed but typically generate warnings.

6.5.5 Unconnected Ports

- Ports may be intentionally left unconnected.
- This is useful for debug signals or unused outputs.

Incorrect port connections, such as connecting an output port to a register externally, result in errors.

6.6 6. Port Connection Methods

Verilog provides two methods for connecting module ports to external signals during instantiation.

6.6.1 Ordered List Connection

- Signals are connected based on the order of ports in the module definition.
- The sequence of signals must exactly match the port list.

This method is simple but error-prone for large modules.

6.6.2 Named Connection

- Signals are connected explicitly using port names.
- The order of connections does not matter.
- Only required ports need to be specified.

This method is preferred for large designs due to improved readability and reduced errors.

6.7 7. Hierarchical Name Referencing

Verilog supports hierarchical design, where modules are instantiated within other modules. Each identifier in the design can be uniquely referenced using a hierarchical name.

6.7.1 Key Concepts

- The top-level module is called the root module.
- Each instance and signal has a unique position in the hierarchy.
- Hierarchical names are constructed using dot notation.

Example structure: - top_module.instance_name.signal_name

6.7.2 Benefits

- Allows access to signals across different levels of hierarchy.
- Enables debugging and monitoring of internal signals.

6.7.3 Special Usage

- The %m format specifier in $display can be used to print the hierarchical name during simulation.

---

7 Gate-Level Modeling in Verilog

7.1 1. Overview of Gate-Level Modeling

Gate-level modeling describes digital circuits using logic gates such as AND, OR, and NOT. It operates at a lower level of abstraction than RTL and provides a direct correspondence between circuit diagrams and Verilog code.

This modeling style is intuitive for designers familiar with digital logic because each Verilog construct maps closely to physical hardware components. While switch-level modeling is even lower, it is rarely used due to complexity.

7.2 2. Gate Primitives in Verilog

Verilog provides predefined gate primitives that can be instantiated directly without defining modules.

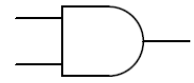
7.2.1 Categories of Gates

7.2.1.1 And/Or Type Gates

- These gates have one output and one or more inputs.
- The first terminal in the instantiation is always the output, followed by inputs.
- Output updates whenever any input changes.

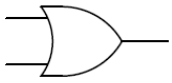
Supported gates include: - and, or, xor - nand, nor, xnor

These gates can accept multiple inputs, and their outputs are computed by applying logic iteratively across all inputs.



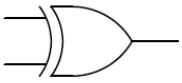
AND

A	B	Output
0	0	0
0	1	0
1	0	0
1	1	1



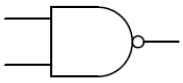
OR

A	B	Output
0	0	0
0	1	1
1	0	1
1	1	1



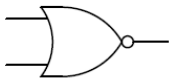
XOR

A	B	Output
0	0	0
0	1	1
1	0	1
1	1	0



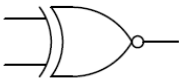
NAND

A	B	Output
0	0	1
0	1	1
1	0	1
1	1	0



NOR

A	B	Output
0	0	1
0	1	0
1	0	0
1	1	0



XNOR

A	B	Output
0	0	1
0	1	0
1	0	0
1	1	1

Figure 6: Logic Gates

7.2.1.2 Buf/Not Type Gates

- These gates have one input and one or more outputs.
- The last terminal represents the input, while preceding terminals are outputs.

Supported gates include: - buf, which passes the input unchanged. - not, which inverts the input.

7.2.1.3 Controlled Gates (Bufif/Notif)

- These gates include an additional control signal.
- They drive the output only when the control signal is active.
- When inactive, the output is high impedance (z).

Supported gates include: - bufif1, bufif0 - notif1, notif0

These gates are useful in designs where multiple drivers share a common signal line.

7.3 3. Gate Instantiation

- Gate primitives are instantiated similarly to modules.
- Instance names are optional for primitives, allowing concise descriptions.
- Multiple inputs or outputs can be specified directly in the instantiation.

Example concepts: - Gates can be used to directly translate logic diagrams into Verilog. - No separate module definition is required for primitive gates.

```
wire OUT, IN1, IN2;

// basic gate instantiations.
and a1(OUT, IN1, IN2);
nand na1(OUT, IN1, IN2);
or or1(OUT, IN1, IN2);
nor nor1(OUT, IN1, IN2);
xor x1(OUT, IN1, IN2);
xnor nx1(OUT, IN1, IN2);
buf b1(OUT1, IN);
not n1(OUT1, IN);
bufif1 b1 (out, in, ctrl);
bufif0 b0 (out, in, ctrl);
notif1 n1 (out, in, ctrl);
notif0 n0 (out, in, ctrl);

// More than two inputs; 3 input nand gate
nand na1_3inp(OUT, IN1, IN2, IN3);
buf b1_2out(OUT1, OUT2, IN);

// gate instantiation without instance name
and (OUT, IN1, IN2); // legal gate instantiation
not (OUT1, IN); // legal gate instantiation
```

7.4 4. Arrays of Gate Instances

Verilog allows creation of arrays of gate instances to simplify repetitive designs.

```
wire [3:0] OUT, IN1, IN2;

// basic gate instantiations.
nand n_gate[3:0](OUT, IN1, IN2);

// This is equivalent to the following 4 instantiations
nand n_gate0(OUT[0], IN1[0], IN2[0]);
nand n_gate1(OUT[1], IN1[1], IN2[1]);
```

```
nand n_gate2(OUT[2], IN1[2], IN2[2]);
nand n_gate3(OUT[3], IN1[3], IN2[3]);
```

- This is useful when performing identical operations across vectors.
- Each instance operates on corresponding bits of input and output vectors.

This approach significantly reduces code duplication and improves readability.

7.5 5. Gate Delays

Real hardware exhibits propagation delays, which can be modeled in Verilog.

7.5.1 Types of Delays

7.5.1.1 Rise Delay

- Represents the time taken for output to transition to logic 1.

7.5.1.2 Fall Delay

- Represents the time taken for output to transition to logic 0.

7.5.1.3 Turn-Off Delay

- Represents the time taken for output to transition to high impedance (z).

If the output transitions to an unknown value (x), the minimum of the specified delays is used.

7.6 6. Delay Specification

Verilog allows different levels of delay specification:

- A single delay value applies to all transitions.
- Two delay values specify rise and fall delays.
- Three delay values specify rise, fall, and turn-off delays explicitly.
- If no delay is specified, the default is zero.

This flexibility allows designers to model timing behavior with varying levels of detail.

7.7 7. Min, Typical, and Max Delays

Each delay type (rise, fall, turn-off) can include three values:

- Minimum delay represents the best-case scenario.
- Typical delay represents nominal behavior.
- Maximum delay represents worst-case conditions.

7.7.1 Usage

- One of these values is selected during simulation.
- Selection is typically controlled via simulator options.

This feature allows designers to analyze circuit behavior under different process variations without modifying the design.

```
// Delay of delay_time for all transitions
and #(delay_time) a1(out, i1, i2);

// Rise and Fall Delay Specification.
and #(rise_val, fall_val) a2(out, i1, i2);

// Rise, Fall, and Turn-off Delay Specification
and #(rise_val, fall_val, turnoff_val) a3(out, i1, i2);

// Three delays - min max & typical
// if +mindelays, rise= 2 fall= 3 turn-off = 4
// if +typdelays, rise= 3 fall= 4 turn-off = 5
// if +maxdelays, rise= 4 fall= 5 turn-off = 6
and #(2:3:4, 3:4:5, 4:5:6) a3(out, i1,i2);
```

7.8 8. Timing Behavior and Simulation

- Gate delays affect the timing of signal transitions in simulation.
- Outputs do not change immediately but follow specified delays.
- Intermediate signals also exhibit delays, impacting overall circuit timing.

Example insight: - A change in input propagates through gates sequentially, each adding its delay. - This results in observable timing differences between signals in waveforms.

---

8 Dataflow Modeling in Verilog

8.1 1. Overview of Dataflow Modeling

Dataflow modeling describes digital circuits in terms of how data moves and is transformed, rather than explicitly instantiating gates.

- It operates at a higher abstraction level compared to gate-level modeling.
- It is widely used in modern design because logic synthesis tools can automatically convert dataflow descriptions into gate-level implementations.
- In practice, designers combine dataflow and behavioral modeling to create RTL (Register Transfer Level) designs.
- This approach improves productivity by allowing designers to focus on functionality and data movement instead of low-level hardware details.

8.2 2. Continuous Assignments

Continuous assignments are the fundamental construct in dataflow modeling and are used to drive values onto nets.

8.2.1 Key Characteristics

- Defined using the assign keyword.
- The left-hand side must always be a net (e.g., wire), not a register.
- The assignment is continuously active and updates whenever any operand on the right-hand side changes.
- The right-hand side can include nets, registers, constants, or function calls.
- Optional delays can be specified to model timing behavior.

Example

```
assign out = in1 & in2;
```

This continuously updates out whenever in1 or in2 changes.

8.3 3. Implicit Continuous Assignment

Verilog allows combining net declaration and assignment into a single statement.

Example

```
wire out = in1 & in2;
```

This is equivalent to declaring a wire and assigning it using a separate assign statement.

8.3.1 Key Point

- Only one implicit assignment is allowed per net because a net can be declared only once.

8.4 4. Implicit Net Declaration

If a signal appears on the left-hand side of an assignment and is not declared, Verilog implicitly declares it as a net.

Example

```
assign out = in1 & in2;
```

Here, out is automatically treated as a wire if not previously declared.

8.5 5. Delays in Continuous Assignments

Delays define when the assigned value appears on the output after input changes.

8.5.1 5.1 Regular Assignment Delay

- Delay is specified in the assign statement.

```
assign #10 out = in1 & in2;
```

- The output updates after 10 time units.
- Uses inertial delay, meaning short pulses (shorter than delay) are ignored.

8.5.2 5.2 Implicit Assignment Delay

- Delay is specified during net declaration.

```
wire #10 out = in1 & in2;
```

8.5.3 5.3 Net Declaration Delay

- Delay applies to any assignment to the net.

```
wire #10 out;
assign out = in1 & in2;
```

8.5.4 Key Insight

- All three methods achieve similar timing behavior but differ in syntax and scope.

8.6 6. Expressions, Operands, and Operators

Dataflow modeling relies heavily on expressions.

8.6.1 Expressions

- Combinations of operands and operators that evaluate to a value.

Example:

```
out = a ^ b;
```

8.6.2 Operands

- Inputs to expressions, which can include:
  - Constants
  - Nets and registers
  - Bit-selects and part-selects
  - Function calls

8.6.3 Operators

- Symbols that perform operations on operands.

8.7 7. Operator Types in Verilog

Verilog provides a wide range of operators to model complex logic.

8.7.1 7.1 Arithmetic Operators

- Perform mathematical operations such as addition, subtraction, multiplication, division, modulus, and exponentiation.
- If any operand contains unknown (x), the result becomes unknown.

Operator Type	Operator Symbol	Operation Performed	Number of Operands
Arithmetic	$*$	Multiply	Two
Arithmetic	/	Divide	Two
Arithmetic	+	Add	Two
Arithmetic	−	Subtract	Two
Arithmetic	%	Modulus	Two
Arithmetic	$**$	Power (Exponent)	Two

8.7.2 7.2 Logical Operators

- Include logical AND (&&), OR (||), and NOT (!).
- Always produce a 1-bit result (0, 1, or x).
- Any non-zero value is treated as true.

Operator Type	Operator Symbol	Operation Performed	Number of Operands
Logical	!	Logical Negation	One
Logical	&&	Logical AND	Two
Logical	||	Logical OR	Two

8.7.3 7.3 Relational Operators

- Compare values using operators such as >, <, >=, <=.
- Result is 1 if true, 0 if false, and x if unknown values are involved.

Operator Type	Operator Symbol	Operation Performed	Number of Operands
Relational	>	Greater Than	Two
Relational	<	Less Than	Two
Relational	>=	Greater Than or Equal	Two
Relational	<=	Less Than or Equal	Two

8.7.4 7.4 Equality Operators

Operator Type	Operator Symbol	Operation Performed	Number of Operands
Equality	==	Equality	Two

Operator Type	Operator Symbol	Operation Performed	Number of Operands
Equality	!=	Inequality	Two
Equality	===	Case Equality	Two
Equality	!==	Case Inequality	Two

Key difference: * Logical equality returns x if operands contain unknowns. * Case equality compares exact bit patterns including x and z.

8.7.5 7.5 Bitwise Operators

Operator Type	Operator Symbol	Operation Performed	Number of Operands
Bitwise	~	Bitwise Negation	One
Bitwise	&	Bitwise AND	Two
Bitwise	|	Bitwise OR	Two
Bitwise	^	Bitwise XOR	Two
Bitwise	^~ or ~^	Bitwise XNOR	Two

8.7.6 7.6 Reduction Operators

- Operate on all bits of a single operand and produce a 1-bit result.

Operator Type	Operator Symbol	Operation Performed	Number of Operands
Reduction	&	Reduction AND	One
Reduction	~&	Reduction NAND	One
Reduction	|	Reduction OR	One
Reduction	~|	Reduction NOR	One
Reduction	^	Reduction XOR	One
Reduction	^~ or ~^	Reduction XNOR	One

Example:

```
parity = ^data; // XOR of all bits
```

8.7.7 7.7 Shift Operators

- Shift bits left or right.
- Arithmetic shifts preserve sign for signed data.

Operator Type	Operator Symbol	Operation Performed	Number of Operands
Shift	>>	Right Shift	Two
Shift	<<	Left Shift	Two
Shift	>>>	Arithmetic Right Shift	Two
Shift	<<<	Arithmetic Left Shift	Two

8.7.8 7.8 Concatenation Operator

- Combines multiple signals into a single vector.

Operator Type	Operator Symbol	Operation Performed	Number of Operands
Concatenation	{ }	Concatenation	Any Number

Example:

```
assign {carry, sum} = a + b;
```

8.7.9 7.9 Replication Operator

- Repeats a value multiple times.

Operator Type	Operator Symbol	Operation Performed	Number of Operands
Replication	{{ }}	Replication	Any Number

Example:

```
out = {4{1'b1}}; // 1111
```

8.7.10 7.10 Conditional Operator

- Acts like a multiplexer.
- If condition is unknown, both branches are evaluated and compared bitwise.

Operator Type	Operator Symbol	Operation Performed	Number of Operands
Conditional	?:	Conditional	Three

Example:

```
assign out = sel ? in1 : in0;
```

8.8 8. Operator Precedence

- Operators follow a defined precedence order, from unary operators (highest) to conditional operator (lowest).
- Parentheses should be used to avoid ambiguity and improve readability.

---

9 Behavioral Modeling in Verilog

9.1 1. Overview of Behavioral Modeling

Behavioral modeling describes a digital system in terms of its functionality and algorithm rather than its structural implementation.

- It operates at the highest level of abstraction in Verilog.
- Designers focus on algorithm behavior and system performance instead of gates or data paths.
- It is commonly used during architectural exploration before RTL implementation.
- The coding style resembles high-level programming languages such as C.
- Behavioral constructs provide flexibility to model complex control logic efficiently.

9.2 2. Structured Procedures

Behavioral code must be written inside structured procedural blocks.

9.2.1 `initial` Block

- Executes once, starting at simulation time 0, and then terminates.
- Multiple `initial` blocks run concurrently.
- Commonly used for:
  - Initialization
  - Testbench stimulus
  - Simulation control

Example:

```
initial begin
  a = 0;
  #10 b = 1;
end
```

9.2.2 `always` Block

- Executes continuously in a loop starting at time 0.
- Used to model ongoing hardware behavior such as clocks or sequential logic.

Example:

```
always #10 clk = ~clk;
```

9.2.3 Key Difference

- `initial` runs once, while always runs forever.

9.3 3. Procedural Assignments

Procedural assignments update variables such as reg, integer, or time.

9.3.1 3.1 Blocking Assignments (=)

- Executed sequentially within a block.
- Each statement completes before the next begins.
- Suitable for combinational logic modeling.

Example:

```
a = b;
c = a; // uses updated value of a
```

9.3.2 3.2 Nonblocking Assignments (<=)

- Schedule updates without blocking subsequent statements.
- All right-hand side values are evaluated first, then updates occur.
- Used for modeling synchronous (clocked) logic.

Example:

```
a <= b;
c <= a; // uses old value of a
```

9.3.3 Key Insight

- Blocking assignments are sequential.
- Nonblocking assignments model parallel updates and avoid race conditions.

9.4 4. Timing Controls

Timing control defines when statements execute in simulation.

9.4.1 4.1 Delay-Based Timing Control

9.4.1.1 Regular Delay

- Delays execution of the entire statement.

```
#10 a = b;
```

9.4.1.2 Intra-Assignment Delay

- Evaluates RHS immediately but delays assignment.

```
a = #5 b;
```

9.4.1.3 Zero Delay

- Ensures execution occurs at the end of the current time step.
- Used to manage race conditions but generally discouraged.

9.4.2 4.2 Event-Based Timing Control

Triggered by signal changes.

```
@(posedge clk) q = d;
```

9.4.2.2 Named Events

- Custom events can be triggered and detected.

```
always @(a or b or c)
```

```
always @(*)
```

- Automatically includes all signals used in the block.
- Recommended for combinational logic.

9.4.3 4.3 Level-Sensitive Timing Control

- Waits for a condition to become true.

```
wait (enable) count = count + 1;
```

9.5 5. Conditional Statements

Used for decision-making in behavioral code.

9.5.1 Types

```
if (enable) out = in;
```

```
if (a > b)
  max = a;
else
  max = b;
```

9.5.1.3 Nested If-Else

- Used for multiple conditions.

9.5.2 Key Behavior

- Condition evaluates to true (non-zero) or false (zero or unknown).

9.6 6. Case Statements (Multiway Branching)

Used when multiple conditions depend on a single expression.

9.6.1 Basic Case

```
case(sel)
  2'd0: out = a;
  2'd1: out = b;
  default: out = 0;
endcase
```

- Acts like a multiplexer.
- Only one branch executes.

9.6.2 Variants

9.6.2.1 `casez`

- Treats z as don't care.

9.6.2.2 `casex`

- Treats both x and z as don't care.

9.7 7. Looping Constructs

Loops allow repetitive execution inside procedural blocks.

9.7.1 7.1 While Loop

- Executes while condition is true.

```
while(i < 10)
  i = i + 1;
```

9.7.2 7.2 For Loop

- Includes initialization, condition, and increment.

```
for(i = 0; i < 10; i = i + 1)
  sum = sum + i;
```

9.7.3 7.3 Repeat Loop

- Executes fixed number of times.

```
repeat(8)
  data = data + 1;
```

9.7.4 7.4 Forever Loop

- Infinite loop, typically used with timing control.

```
forever #5 clk = ~clk;
```

9.8 8. Sequential vs Parallel Blocks

9.8.1 Sequential Block (`begin-end`)

- Statements execute in order.

```
begin
  a = 1;
  b = a;
end
```

9.8.2 Parallel Block (`fork-join`)

- Statements execute concurrently.

```
fork
  #5 a = 1;
  #10 b = 1;
join
```

9.8.3 Key Insight

- Sequential blocks ensure order.
- Parallel blocks allow concurrency but may introduce race conditions.

9.9 9. Named Blocks and Control

9.9.1 Named Blocks

- Blocks can be named for hierarchy and control.

```
begin : block1
```

9.9.2 Disable Statement

- Terminates execution of a named block.

```
disable block1;
```

- Useful for breaking loops or early exits.

9.10 10. Generate Blocks (Elaboration-Time Constructs)

Generate constructs create hardware structures before simulation begins.

9.10.1 Key Characteristics

- Evaluated at compile/elaboration time.
- Used for parameterized and scalable designs.

9.10.2 10.1 Generate Loop

- Repeats structures using genvar.

Example:

```
generate
  for(i=0; i<N; i=i+1)
    assign out[i] = a[i] ^ b[i];
endgenerate
```

9.10.3 10.2 Generate Conditional

- Instantiates different hardware based on parameters.

```
generate
  if (WIDTH < 8)
    small_unit u1(...);
  else
    large_unit u2(...);
endgenerate
```

9.10.4 10.3 Generate Case

- Selects implementation based on parameter values.

---

10 Tasks and Functions in Verilog

10.1 1. Overview

Tasks and functions are used to modularize and reuse commonly used behavioral code in Verilog designs.

- They help reduce code duplication and improve readability and maintainability.
- Both are defined within a module and are part of the design hierarchy.
- They are invoked from initial , always, or other tasks/functions.
- They contain only behavioral statements and cannot include always or initial blocks.

10.2 2. Tasks vs Functions

10.2.1 Functional Distinction

- A **function** is used for combinational logic that:
  - Executes in zero simulation time.
  - Returns exactly one value.
  - Has only input arguments.
- A **task** is used for more general procedures that:
  - May include delays, events, or timing control.
  - Can return multiple values via output/inout arguments.
  - Can have zero or more arguments.

10.2.2 Summary of Differences

Feature	Function	Task
Execution time	Zero time	May consume time
Timing control	Not allowed	Allowed
Return value	Single value	No direct return, multiple outputs allowed
Arguments	Only inputs	Input, output, inout
Invocation capability	Can call functions only	Can call both tasks and functions

---

10.3 3. Tasks

10.3.1 3.1 When to Use Tasks

- The logic includes delays or timing control.

- Multiple outputs are required.
- There are no input arguments.

10.3.2 3.2 Task Declaration and Invocation

Tasks are declared using task and endtask.

- Arguments are passed in positional order.
- Output values are returned when the task completes.
- Tasks can call other tasks and functions.

Example:

```
task bitwise_op;
  input [7:0] a, b;
  output [7:0] out;
  begin
    out = a & b;
  end
endtask
```

Invocation:

```
bitwise_op(A, B, OUT);
```

10.3.3 3.3 Task with Timing Control

Tasks can include delays, making them suitable for sequential behavior.

Example:

```
task delayed_and;
  input a, b;
  output out;
  begin
    #10 out = a & b;
  end
endtask
```

10.3.4 3.4 Automatic (Re-entrant) Tasks

- By default, tasks are **static**, meaning variables are shared across calls.
- This can cause incorrect behavior if tasks are invoked concurrently.
- Each call gets its own independent variable space.
- Recommended when tasks may be called concurrently.

To solve this, use automatic:

```
task automatic compute;
```

10.4 4. Functions

10.4.1 4.1 When to Use Functions

A function must satisfy all of the following:

- No delay or timing control.
- At least one input argument.
- Returns exactly one value.
- No output or inout arguments.
- No nonblocking assignments.

10.4.2 4.2 Function Declaration and Return Mechanism

Functions use an implicit variable (same name as function) to return values. * Return value is assigned to the function name. * Default return width is 1 bit unless specified.

Example:

```
function parity;
  input [7:0] data;
  begin
    parity = ^data;
  end
endfunction
```

Invocation:

```
p = parity(data);
```

10.4.3 4.3 Function with Explicit Width

Example:

```
function [31:0] shift;
  input [31:0] data;
  input dir;
  begin
    shift = (dir) ? (data >> 1) : (data << 1);
  end
endfunction
```

10.4.4 4.4 Automatic (Recursive) Functions

- Functions can be declared automatic for recursion or concurrent calls.
- Each call gets independent storage.
- Enables recursion and safe concurrent usage.

Example:

```
function automatic integer factorial;
  input integer n;
  begin
    if (n <= 1)
      factorial = 1;
    else
      factorial = n * factorial(n-1);
  end
endfunction
```

10.5 5. Constant Functions

- Used to compute values at compile/elaboration time.
- Commonly used for parameter calculations.

Example:

```
function integer clog2;
  input integer depth;
  begin
    for(clog2 = 0; depth > 0; clog2 = clog2 + 1)
      depth = depth >> 1;
  end
endfunction
```

Usage:

```
input [clog2(256)-1:0] addr;
```

10.6 6. Signed Functions

- Functions can return signed values using signed.
- Enables signed arithmetic comparisons and operations.

Example:

```
function signed [31:0] compute;
```

10.7 7. Best Practices

- Use **functions** for combinational logic and calculations.
- Use **tasks** for:
  - Sequential behavior
  - Delays and timing control
  - Multiple outputs
- Avoid using static tasks when concurrent execution is possible; prefer automatic.

- Keep functions simple and side-effect free.
- Ensure correct argument ordering during task invocation.
- Use functions for reusable logic like parity, encoding, or arithmetic operations.

---

11 Useful Modeling Techniques in Verilog

11.1 1. Overview

This chapter covers advanced Verilog features that improve modeling flexibility, debugging capability, parameter reuse, simulation control, and verification efficiency.

Main topics include:

- Procedural continuous assignments
- Parameter overriding
- Conditional compilation and execution
- Time scales
- Useful system tasks for files, hierarchy, randomness, memory loading, and waveform dumps :contentReferenceoaicite:0

11.2 2. Procedural Continuous Assignments

These statements continuously drive values onto registers or nets for a limited time and temporarily override normal assignments.

11.3 2.1 `assign` and `deassign`

Used only with registers.

11.3.1 Behavior

- assign forces a register to continuously follow an expression.
- deassign removes that override.
- After deassign, the register keeps its last value until changed by another procedural assignment.

11.3.2 Typical Use

Historically used for asynchronous overrides such as reset logic.

11.3.3 Example

```
if(reset)
  assign q = 1'b0;
else
  deassign q;
```

11.3.4 Important Note

- Considered outdated coding style in modern RTL.
- Prefer explicit synchronous or asynchronous logic in always blocks.

11.4 2.2 `force` and `release`

Used on both registers and nets.

11.4.1 Behavior

- force overrides all existing drivers.
- release removes the override.

11.4.2 Common Use

- Debugging
- Testbench stimulus
- Fault injection

11.4.3 Example

```
#50 force dut.q = 1'b1;
#50 release dut.q;
```

11.4.4 Net Behavior

If used on a net, normal driven value resumes immediately after release.

11.4.5 Best Practice

Use in testbenches, not inside synthesizable design logic.

11.5 3. Overriding Parameters

Parameters make modules configurable. Values can be changed per instance.

11.6 3.1 Using `defparam`

Overrides parameters through hierarchical names.

11.6.1 Example

```
defparam u1.WIDTH = 8;
```

11.6.2 Important Note

- Legal Verilog, but considered poor style.
- Avoid in modern code.

11.7 3.2 Parameter Override at Instantiation

Preferred modern method.

11.7.1 Ordered Form

```
adder #(8) u1 (...);
```

11.7.2 Named Form (Recommended)

```
adder #(.WIDTH(8)) u1 (...);
```

11.7.3 Why Named Form is Better

- Clearer intent
- Safer when parameter order changes
- Easier maintenance

11.8 4. Conditional Compilation

Used to include or exclude code during compilation.

11.8.1 Directives

- `ifdef
- `ifndef
- `elsif
- `else
- `endif

11.8.2 Example

```
`ifdef DEBUG
  initial $display("Debug mode");
`endif
```

11.8.3 Typical Uses

- Debug builds
- Feature enable/disable
- Environment-specific code
- Simulation-only logic

11.8.4 Key Note

These are compile-time controls, not run-time controls.

11.9 5. Conditional Execution at Run Time

All code is compiled, but selected statements execute only when runtime flags are passed.

11.10 5.1 `$test$plusargs`

Checks whether a runtime option exists.

11.10.1 Example

```
if($test$plusargs("DEBUG"))
  $display("Debug enabled");
```

Run simulator with:

```
+DEBUG
```

11.11 5.2 `$value$plusargs`

Reads runtime values.

11.11.1 Example

```
integer period;
$value$plusargs("clk_t=%d", period);
```

Run simulator with:

```
+clk_t=10
```

11.11.2 Typical Uses

- Clock period selection
- Testcase names
- Mode selection
- Random seeds

11.12 6. Time Scales

Defines time unit and simulation precision.

11.12.1 Syntax

```
`timescale <time_unit> / <precision>
```

11.12.2 Example

```
`timescale 1ns / 1ps
```

11.12.3 Meaning

- 1ns = delay unit for #1
- 1ps = rounding precision

11.12.4 Impact

#5

means:

- 5 ns if timescale is 1ns
- 5 us if timescale is 1us

11.12.5 Best Practice

Use consistent timescales across project files.

11.13 7. Useful System Tasks

11.14 7.1 File Output

Used to log simulation data to files.

11.14.1 Open File

```
fd = $fopen("log.txt");
```

11.14.2 Write to File

```
$fdisplay(fd, "count=%0d", count);
```

11.14.3 Close File

```
$fclose(fd);
```

11.14.4 Related Tasks

- $fdisplay
- $fwrite
- $fmonitor
- $fstrobe

Useful for automated regression logs.

11.15 7.2 Displaying Hierarchy with %m

Prints current hierarchical scope.

11.15.1 Example

```
$display("Running in %m");
```

Possible output:

```
top.u1.alu
```

Useful when multiple identical instances exist.

11.16 7.3 $strobe

Displays values after all updates in the current time step complete.

11.16.1 Example

```
$strobe("q=%b", q);
```

11.16.2 Difference from $display

- $display may print before scheduled assignments finish.
- $strobe prints final settled values for that timestep.

Useful in race-sensitive debugging.

11.17 7.4 Random Number Generation

11.17.1 Task

$random

11.17.2 Example

```
addr = $random(seed);
```

11.17.3 Notes

- Returns signed 32-bit integer.
- Seed gives repeatable sequences.

Useful for randomized testbenches.

11.18 7.5 Memory Initialization from File

Loads ROM/RAM contents from external files.

11.18.1 Binary File

```
$readmemb("init.mem", mem);
```

11.18.2 Hex File

```
$readmemh("init.hex", mem);
```

11.18.3 Example Use

```
reg [7:0] mem [0:255];
```

Useful for:

- Program memory
- Lookup tables
- Test vectors

11.19 7.6 Value Change Dump (VCD)

Creates waveform dump files for viewing signal activity.

11.19.1 Specify File

```
$dumpfile("wave.vcd");
```

11.19.2 Dump Signals

```
$dumpvars;
```

11.19.3 Control Dumping

```
$dumpon;
$dumpoff;
$dumpall;
```

11.19.4 Uses

- Debugging waveforms
- Timing inspection
- Post-simulation analysis

11.19.5 Important Note

VCD files can become very large. Dump only required signals.

11.20 8. Best Practices

- Prefer parameter override during instantiation instead of defparam.
- Use force/release only in testbench or debug environments.
- Use named parameters for readability.
- Use $value$plusargs for configurable simulations.
- Keep timescales consistent.
- Use waveform dumps selectively to control file size.
- Prefer file logging for long regressions instead of console-only output.

11.21 Key Takeaways

- Procedural continuous assignments temporarily override normal drivers.
- Parameterization enables reusable and scalable modules.
- Conditional compilation and plusargs improve environment flexibility.
- timescale controls delay interpretation and precision.
- System tasks greatly improve debugging, logging, memory loading, and waveform analysis.
- These techniques are widely used in professional verification and simulation workflows.

12 Logic Synthesis with Verilog HDL

12.1 1. Overview of Logic Synthesis

Logic synthesis is the automated process of converting a high-level hardware description, usually RTL Verilog, into an optimized gate-level netlist using a target technology library and design constraints.

12.1.1 Core Idea

The designer writes behavior and data-transfer logic at RTL level. The synthesis tool converts that description into gates, flip-flops, multiplexers, arithmetic cells, and other hardware primitives supported by the fabrication library.

12.1.2 Inputs to Logic Synthesis

- RTL Verilog source code

- Technology library (standard cells/macros)
- Constraints such as timing, area, and power
- Operating conditions such as loads and drive strengths

12.1.3 Output

- Gate-level netlist mapped to target library cells

12.1.4 Why It Matters

Before synthesis tools, designers manually converted logic into schematics. This was slower, error-prone, difficult to optimize, and hard to retarget to another fabrication process. :contentReferenceoaicite:0

12.2 2. Benefits and Industry Impact of Logic Synthesis

Logic synthesis significantly improved productivity and reduced design cycle time.

12.2.1 Major Advantages

12.2.1.1 Faster Development Tasks that previously took months can often be completed in hours or days.

12.2.1.2 Higher Abstraction Design Designers focus on functionality and architecture instead of drawing gates manually.

12.2.1.3 Easier Iteration If requirements change, modify RTL and resynthesize instead of redesigning gate schematics.

12.2.1.4 Better Optimization Tools optimize globally across the design for timing, area, and power.

12.2.1.5 Technology Independence The same RTL can target different fabrication libraries.

12.2.1.6 Improved Reuse Reusable RTL IP blocks can be used across products and technologies.

12.2.1.7 Easier What-If Analysis Example: - Same RTL with 20 ns clock target - Re-run synthesis for 15 ns target

No need to redesign by hand.

12.3 3. What Logic Synthesis Actually Does

Logic synthesis converts RTL into hardware in multiple internal stages.

12.4 3.1 Translation

The tool reads RTL constructs and converts them into an internal intermediate representation.

Example:

```
assign y = (a & b) | c;
```

Interpreted as combinational Boolean logic.

12.5 3.2 Logic Optimization

The tool removes redundant logic and simplifies Boolean expressions.

Goals:

- Fewer gates
- Better speed
- Lower power

12.6 3.3 Technology Mapping

The optimized logic is implemented using cells available in the target library.

Example library cells:

- NAND
- NOR
- AND
- OR
- Inverter
- D Flip-Flop
- Multiplexer
- Adder

12.7 3.4 Technology-Dependent Optimization

After mapping, the tool performs local optimization using real cell delays, area, and power data.

12.8 4. Technology Library (Standard Cell Library)

A technology library is a collection of cells provided by a semiconductor vendor.

Each cell includes:

- Functional behavior
- Area
- Timing characteristics

- Power data
- Pin information

12.8.1 Example Cells

- 2-input NAND
- Inverter
- Positive-edge DFF
- Adder macro
- Mux cells

12.8.2 Important Insight

Synthesis quality strongly depends on library quality. A limited library restricts optimization possibilities.

12.9 5. Design Constraints

Synthesis requires realistic constraints to produce useful hardware.

12.10 5.1 Timing Constraints

Specify required clock period or path timing.

Example:

- 5 ns clock means faster logic required.

12.11 5.2 Area Constraints

Specify maximum silicon area.

12.12 5.3 Power Constraints

Specify power limits.

12.13 5.4 Environmental Constraints

Include:

- Input arrival time
- Output load
- Drive strength
- Operating corners

12.13.1 Trade-Off: Area vs Speed

Faster circuits often require:

- More parallel logic
- Larger area

Smaller circuits often require:

- Slower implementations

12.14 6. RTL for Logic Synthesis

Most synthesis flows use Register Transfer Level (RTL) descriptions.

RTL combines:

- Dataflow modeling
- Behavioral modeling
- Clocked sequential logic

Example:

```
always @(posedge clk)
q <= d;
```

This typically infers a D flip-flop.

12.15 7. Synthesizable Verilog Constructs

Not every Verilog feature is synthesizable.

12.16 Commonly Accepted Constructs

12.16.1 Module Structure

- module, endmodule

12.16.2 Ports

- input
- output
- inout

12.16.3 Signals

- wire
- reg
- vectors

12.16.4 Continuous Assignment

```
assign y = a & b;
```

12.16.5 Procedural Logic

- always
- if
- else
- case

12.16.6 Loops

- for

12.16.7 Tasks / Functions

Supported if synthesizable style is used.

12.16.8 Module Instantiation

Reusable submodules may be instantiated.

12.17 8. Common Non-Ideal / Unsupported Constructs

12.18 8.1 `initial`

Usually not synthesizable for ASIC flows.

Use reset logic instead.

12.19 8.2 Delays

```
#10 a = b;
```

Ignored by synthesis.

12.20 8.3 X/Z Comparison Operators

Often not meaningful in synthesis:

- ===
- !==

12.21 8.4 Infinite Combinational Loops

Unsafe or invalid hardware intent.

12.22 9. How Synthesis Interprets Verilog Code

12.23 9.1 Continuous Assignments Infer Combinational Logic

```
assign out = (a & b) | c;
```

Creates AND + OR logic.

12.24 9.2 Conditional Operator Infers Multiplexer

```
assign y = sel ? b : a;
```

Infers 2:1 mux.

12.25 9.3 If-Else Often Infers Mux

```
if(sel)
 y = b;
else
 y = a;
```

12.26 9.4 Case Statement Often Infers Multiway Mux

```
case(op)
2'b00: y = a;
2'b01: y = b;
endcase
```

12.27 9.5 Clocked Always Block Infers Flip-Flop

```
always @(posedge clk)
q <= d;
```

Positive-edge triggered DFF.

12.28 9.6 Incomplete Combinational If/Case Infers Latch

```
always @(*)
if(en)
 y = a;
```

No else branch means storage is required.

12.29 10. Example: Magnitude Comparator

A 4-bit comparator compares A and B.

12.29.1 Outputs

- A_gt_B
- A_lt_B
- A_eq_B

12.29.2 RTL Example

```
assign A_gt_B = (A > B);
assign A_lt_B = (A < B);
assign A_eq_B = (A == B);
```

12.29.3 Insight

Very compact RTL can synthesize into many gates depending on technology and timing goals.

12.30 11. Verification of Synthesized Netlist

After synthesis, gate-level netlist must be verified.

12.31 11.1 Functional Verification

Apply same testbench to:

- Original RTL
- Synthesized gate-level netlist

Outputs should match logically.

12.32 11.2 Timing Verification

Use:

- Static Timing Analysis (STA)
- Gate-level timing simulation

Check setup/hold and path delays.

12.33 12. Need for Simulation Library

Gate netlists instantiate vendor cells such as:

VAND
VNAND
PDFF

Simulators need behavioral models of these cells.

Therefore vendors provide simulation libraries describing these cells in Verilog.

12.34 13. Coding Style Guidelines for Better Synthesis

RTL style directly affects final hardware quality.

12.35 13.1 Use Meaningful Signal Names

Prefer:

```
data_valid
```

Instead of:

```
x1
```

12.36 13.2 Avoid Mixing Positive and Negative Edge Flops

Can complicate clock tree and create skew issues.

12.37 13.3 Use Parentheses to Control Structure

```
out = (a+b) + (c+d);
```

Can allow more parallel implementation than:

```
out = a+b+c+d;
```

12.38 13.4 Be Careful with Expensive Operators

Operators like:

- *
- /
- %

May create large hardware.

Use only when justified.

12.39 13.5 Avoid Multiple Assignments to Same Register from Separate Blocks

Bad example:

```
always @(posedge clk) if(ld1) q <= a;
always @(posedge clk) if(ld2) q <= b;
```

Creates ambiguous hardware and poor synthesis results.

12.40 13.6 Fully Specify If / Case Statements

Always include else or default unless latch intended.

12.41 14. Design Partitioning for Better Results

Large monolithic blocks may synthesize poorly.

12.42 14.1 Horizontal Partitioning

Split by bit slices.

Example:

- Build 16-bit ALU using four 4-bit ALUs.

12.42.1 Benefit

Smaller optimization problems.

12.43 14.2 Vertical Partitioning

Split by function.

Example ALU into:

- Add unit
- Subtract unit
- Shift unit
- Control mux

12.43.1 Benefit

Clear hierarchy and modular optimization.

12.44 14.3 Parallelization

Use more hardware to improve speed.

Example:

- Ripple carry adder = smaller but slower
- Carry lookahead adder = larger but faster

12.45 15. Sequential Circuit Synthesis Example – FSM

Example system: newspaper vending machine.

12.45.1 Rules

- Cost = 15 cents
- Accepts nickels and dimes
- Releases newspaper when enough money inserted

12.45.2 Inputs

```
00 = no coin
01 = nickel
10 = dime
```

12.45.3 Output

```
newspaper = 1
```

for one clock cycle when dispense occurs.

12.46 16. FSM States

States represent accumulated amount:

- s0 = 0 cents
- s5 = 5 cents
- s10 = 10 cents
- s15 = vend state

12.46.1 Example Transition

From s10, insert nickel -> s15

Then output newspaper and return to s0.

12.47 17. RTL FSM Implementation Structure

Typical synthesizable FSM has two parts:

12.48 17.1 Combinational Next-State Logic

Determines:

- next state
- output

12.49 17.2 Sequential State Register

```
always @(posedge clock)
  PRES_STATE <= NEXT_STATE;
```

This infers state flip-flops.

12.50 18. Final Gate-Level Netlist

After synthesis, FSM becomes:

- Flip-flops for state bits
- Gates implementing transitions
- Output decode logic

12.51 19. Practical Professional Workflow

1. Write clean RTL.
2. Simulate RTL thoroughly.
3. Apply constraints.
4. Run synthesis.
5. Check reports:
   - timing
   - area
   - power
6. Run equivalence / gate simulation.
7. Iterate RTL or constraints if needed.
8. Handoff for place-and-route.

12.52 20. Critical Professional Insights

- Synthesis does not fix bad architecture.
- Constraints strongly influence output quality.
- Clean RTL often matters more than clever RTL.
- Over-constraining can waste area/power.
- Under-constraining can fail timing.
- Partitioning improves scalability.
- Verification after synthesis is mandatory.

12.53 Key Takeaways

- Logic synthesis converts RTL Verilog into optimized gate-level hardware.
- Inputs are RTL, technology library, and constraints.
- Output quality depends on coding style, architecture, and constraints.
- Continuous assignments infer combinational logic; clocked always blocks infer registers.
- Incomplete combinational logic can infer latches unintentionally.
- Partitioning and hierarchy improve synthesis quality.
- Post-synthesis functional and timing verification are essential.
- Logic synthesis is a core step in modern ASIC and FPGA digital design flows.

13 Advanced Verification Techniques

13.1 1. Overview

As chip complexity increased into million-gate designs, verification became the major bottleneck in hardware development.

- Teams often spent more time verifying than designing.
- Detecting bugs late could cause expensive silicon re-spins.
- Traditional Verilog-only simulation flows became insufficient for large systems.

Modern verification adds methodologies such as:

- Structured verification environments
- Coverage-driven verification
- Assertion-based verification
- Formal and semi-formal verification
- Equivalence checking
- Acceleration and emulation platforms :contentReferenceoaicite:0

13.2 2. Traditional Verification Flow

A typical verification flow follows these steps:

Step	Description
1	Create architecture and design specification.
2	Build functional test plan based on specification.
3	Apply tests to the Design Under Test (DUT).
4	Simulate DUT behavior.
5	Analyze outputs against expected results.
6	Review coverage and close verification goals.
7	Optionally use acceleration, emulation, or assertions to reduce risk.

13.2.1 Key Insight

Verification quality depends heavily on the quality of the specification and test plan.

13.3 3. Architectural Modeling

Used before RTL implementation to explore design trade-offs.

13.3.1 Typical Questions

- Number of processors required
- Memory bandwidth needs
- Which functions should be hardware vs software
- Performance vs cost trade-offs

13.3.2 Common Languages

- C
- C++
- Specialized system modeling languages

13.3.3 Benefit

Problems are found early before expensive RTL development begins.

13.4 4. Functional Verification Environment

Verification is usually divided into three phases.

13.4.1 4.1 Block-Level Verification

- Individual IP/block tested in isolation.
- Usually handled by designers.

13.4.2 4.2 Full-Chip Verification

- Entire SoC/chip tested after integration.
- Ensures all planned features work together.

13.4.3 4.3 Extended Verification

- Focuses on corner cases and long random regressions.
- May continue close to or beyond tape-out.

13.5 5. Directed vs Random Testing

13.5.1 Directed Tests

- Specifically written for known features or scenarios.
- Good for protocol bring-up and deterministic debugging.

13.5.2 Random Tests

- Legal randomized transactions applied automatically.
- Excellent for discovering unexpected corner-case bugs.

13.5.3 Best Practice

Use both methods together.

13.6 6. High-Level Verification Languages (HVLs)

Created because pure HDL testbenches became hard to scale.

13.6.1 Why Needed

Large testbenches became: - Hard to maintain - Difficult to reuse - Weak in abstraction - Poor for automated stimulus generation

13.6.2 HVL Advantages

- Object-oriented programming support
- Random stimulus generation
- Reusable components
- Scoreboards and checkers
- Coverage collection
- Protocol validation

13.6.3 Typical Usage Model

- DUT simulated in Verilog/SystemVerilog simulator.
- Testbench environment runs in HVL framework.

13.7 7. Simulation Methods

Three common execution platforms are used.

13.8 7.1 Software Simulation

Traditional RTL simulation on workstation/server.

13.8.1 Advantages

- Low setup cost
- Flexible debugging
- Best for small to medium designs

13.8.2 Limitation

Too slow for very large regressions.

13.9 7.2 Hardware Acceleration

Synthesizable parts of design mapped to hardware platform.

13.9.1 Benefits

- Often 100x to 1000x faster than software simulation.
- Good for long regression runs.

13.9.2 Limitations

- Expensive tools
- Longer compile/setup time

13.9.3 Best Use Case

Stable RTL with heavy simulation workload.

13.10 7.3 Hardware Emulation

Runs design in near real hardware environment.

13.10.1 Benefits

- Can boot operating systems
- Can run real software stacks
- Enables hardware/software co-verification

13.10.2 Example Uses

- Boot Linux/UNIX on processor design
- Display live video decode
- Run graphics workloads

13.10.3 Limitation

High cost and setup complexity.

13.11 8. Analysis of Simulation Results

Verification must answer:

1. Was output data correct?
2. Was interface protocol followed?

Traditional methods: - Waveform viewing - Manual log inspection

These do not scale well.

13.12 9. Self-Checking Testbenches

Modern environments automatically detect failures.

13.12.1 Main Components

13.12.2 9.1 Data Checker / Scoreboard

Compares expected vs actual outputs.

Example concept:

```
sent packet A -> received packet A
```

If mismatch occurs, report failure immediately.

13.12.3 9.2 Protocol Checker

Ensures timing/handshake rules are followed.

Example:

- req must be followed by ack within N cycles.

13.12.4 Benefit

Thousands of tests can run unattended.

13.13 10. Coverage

Coverage measures verification progress and completeness.

13.14 10.1 Structural Coverage

Measures how RTL code was exercised.

13.14.1 Code Coverage

Checks executed statements/lines/branches.

13.14.2 Toggle Coverage

Checks whether bits changed 0−>1 and 1−>0.

13.14.3 Branch Coverage

Checks whether all decision paths were taken.

13.14.4 Limitation

Good indicator of activity, not complete functionality.

13.15 10.2 Functional Coverage

Measures whether intended design behavior was exercised.

Examples:

- All opcodes tested
- FIFO full and empty cases tested
- Interrupt during transfer tested
- All FSM states visited

13.15.1 Key Advantage

Closer to specification than structural coverage.

13.16 11. Assertion Checking

Assertions are statements describing intended behavior.

13.16.1 Types

13.16.2 Static Assertions

Always true conditions.

Example:

```
FIFO full and empty must never both be 1
```

13.16.3 Temporal Assertions

Timing relationships.

Example:

```
ack must arrive within 5 cycles of req
```

13.16.4 Benefits

- Improves observability
- Finds bugs near source
- Reduces debug time
- Enables automated checking

13.16.5 Note

Assertions are generally ignored by synthesis.

13.17 12. Formal Verification

Uses mathematical proof instead of simulation vectors.

13.17.1 Goal

Prove a property is always true or provide counterexample.

13.17.2 Example Property

```
Grant is never given to two masters simultaneously.
```

13.17.3 Advantages

- Exhaustive state-space exploration
- Finds deep corner cases
- No need for test vectors

13.17.4 Challenges

- State explosion on large designs
- Requires good constraints

13.17.5 Constraint Importance

Too loose:

- Illegal scenarios explored

Too tight:

- Real bugs may be hidden

13.18 13. Semi-Formal Verification

Combines simulation and formal methods.

13.18.1 Flow

1. Run simulations.
2. Capture interesting states.
3. Use formal tools around those states.

13.18.2 Benefit

Better scalability than pure formal while finding deep bugs.

13.19 14. Equivalence Checking

Used after synthesis or place-and-route.

13.19.1 Purpose

Ensure implementation matches RTL functionality.

13.19.2 Comparison

- RTL model
- Gate-level netlist
- Physical netlist

13.19.3 Benefit

Avoids rerunning full RTL test suite on gate netlists.

13.19.4 Key Use

Critical signoff step in ASIC/FPGA flows.

13.20 15. Practical Verification Strategy

For professional projects:

1. Build reusable self-checking testbench.
2. Use assertions early.
3. Run directed tests first.
4. Add constrained random regressions.
5. Track structural + functional coverage.
6. Use formal on control logic and protocols.
7. Use equivalence after synthesis.
8. Use acceleration/emulation for SoC scale.

13.21 Key Takeaways

- Verification dominates effort in modern chip design.
- Simulation alone is insufficient for large systems.
- Coverage, assertions, and self-checking testbenches are essential.
- Formal methods prove correctness mathematically.
- Semi-formal methods improve scalability.
- Equivalence checking ensures synthesized hardware matches RTL.
- Acceleration and emulation are critical for large SoCs and software bring-up.

14 IEEE 1364-2005 Verilog HDL

14.1 1. Overview of IEEE 1364-2005

IEEE 1364-2005 is the final major standalone Verilog HDL language standard before Verilog was incorporated into SystemVerilog (IEEE 1800). It formalized syntax, semantics, simulation behavior, RTL modeling rules, and interoperability used across ASIC and FPGA flows.

14.1.1 Why It Matters

- It became the reference language for classic Verilog RTL design and simulation.
- Most legacy ASIC/FPGA codebases still contain IEEE 1364 style Verilog.
- Understanding it is essential for maintaining existing RTL and gate-level netlists.
- Many synthesis coding guidelines still originate from this standard.

14.1.2 Evolution

Standard	Importance
IEEE 1364-1995	First major Verilog IEEE standard
IEEE 1364-2001	Major usability improvements
IEEE 1364-2005	Clarifications, cleanup, final mature Verilog
IEEE 1800	SystemVerilog superseded Verilog

14.2 2. Core Design Philosophy

Verilog models hardware using concurrent processes, event-driven simulation, and multiple abstraction levels.

14.2.1 Supported Modeling Levels

- Behavioral modeling
- Register Transfer Level (RTL)
- Dataflow modeling
- Gate-level modeling
- Switch-level modeling

14.2.2 Key Principle

Verilog statements may look software-like, but semantics model hardware concurrency.

Example:

```
assign y = a & b;
assign z = c | d;
```

Both statements represent hardware operating simultaneously.

14.3 3. Basic Source Structure

A Verilog design is built from modules.

```
module adder(input a, b, output y);
assign y = a ^ b;
endmodule
```

14.3.1 Module Contains

- Port declarations
- Parameters
- Net/register declarations
- Continuous assignments
- Procedural blocks
- Tasks/functions
- Submodule instances

14.3.2 Notes

- Modules may instantiate other modules.
- Hierarchical design is fundamental.

14.4 4. Lexical Rules

14.5 4.1 Case Sensitivity

Verilog is case-sensitive.

```
clk != Clk != CLK
```

14.6 4.2 Comments

```
// single line
/* multi-line */
```

14.7 4.3 Identifiers

May contain:

- letters
- digits
- _
- $

Cannot start with digit.

14.8 4.4 Escaped Identifiers

Used for special names.

\my−s i g n a l −name

(terminated by whitespace)

14.9 5. Data Values and Logic System

Verilog uses a four-state logic system.

Value	Meaning
0	Logic zero
1	Logic one
x	Unknown
z	High impedance

14.9.1 Why Important

Real hardware may have:

- uninitialized signals
- bus contention
- floating tri-state lines

14.10 6. Net Types

Nets model driven connections.

14.10.1 Common Net Types

- wire
- tri
- wand
- wor
- supply0
- supply1

14.10.2 Key Rule

Nets do not store values. They reflect driven values.

```
wire y;
assign y = a & b;
```

14.11 7. Variable Types

Variables store procedural values.

14.11.1 Common Types

- reg
- integer
- time
- real
- realtime

14.11.2 Important Clarification

reg does **not** automatically mean hardware register. It means procedural assignment variable.

```
reg y;
always @(*) y = a & b;
```

This can synthesize to combinational logic.

14.12 8. Scalars, Vectors, Arrays, Memories

14.13 Scalars

Single bit.

```
reg a;
```

14.14 Vectors

Multi-bit packed signals.

```
reg [7:0] data;
```

14.15 Memories

Array of words.

```
reg [7:0] mem [0:255];
```

256 entries of 8 bits each.

14.16 9. Numbers and Literals

Format:

<size>'<base><value>

Examples:

```
8'b10100101
16'h00FF
4'd9
```

14.16.1 Bases

- b binary
- o octal
- d decimal
- h hexadecimal

14.16.2 Supports x/z digits

```
4'b10xz
```

14.17 10. Operators

14.18 Arithmetic

+ − ∗ / %

14.19 Relational

< <= > >=

14.20 Equality

== !=

14.21 Case Equality

=== !==

Includes x/z matching.

14.22 Logical

! && ||

14.23 Bitwise

~ & | ^ ^~

14.24 Reduction

Unary reduction across vector.

```
&data
|data
^data
```

14.25 Shift

<< >> <<< >>>

14.26 Conditional

```
sel ? a : b
```

Used to infer mux logic.

14.27 11. Continuous Assignments

Used for combinational net driving.

```
assign y = (a & b) | c;
```

14.27.1 Characteristics

- Reevaluates whenever RHS changes.
- LHS usually net type.

14.28 12. Procedural Blocks

Two main procedural processes:

14.29 12.1 `initial`

Runs once at simulation start.

```
initial begin
  reset = 1;
end
```

Mostly simulation/testbench usage.

14.30 12.2 `always`

Runs forever.

```
always #5 clk = ~clk;
```

Used in RTL and testbench.

14.31 13. Event Controls and Sensitivity Lists

Used to trigger procedural execution.

14.32 Level Sensitive

```
always @(a or b or sel)
```

14.33 Edge Sensitive

```
always @(posedge clk)
always @(negedge rst_n)
```

14.34 Recommended Combinational Form

```
always @(*)
```

Automatically infers dependencies.

14.35 14. Blocking vs Nonblocking Assignments

14.36 Blocking =

Sequential execution within block.

```
a = b;
c = a;
```

c gets updated a.

14.37 Nonblocking <=

Scheduled parallel update.

```
a <= b;
c <= a;
```

c gets old a.

14.37.1 Professional Rule

- Use blocking for combinational logic.
- Use nonblocking for sequential clocked logic.

14.38 15. Conditional Statements

14.39 If / Else

```
if(en) y = a;
else y = b;
```

14.40 Case

```
case(sel)
2'b00: y = a;
2'b01: y = b;
default: y = 0;
endcase
```

14.40.1 Variants

- case
- casex
- casez

Use cautiously because wildcards may hide bugs.

14.41 16. Loops

Supported procedural loops:

- for
- while
- repeat
- forever

Example:

```
for(i=0;i<8;i=i+1)
  sum[i] = a[i] ^ b[i];
```

14.42 17. Tasks and Functions

14.43 Functions

- Return one value
- No timing control

```
function parity;
input [7:0] d;
begin
  parity = ^d;
end
endfunction
```

14.44 Tasks

- May have multiple outputs
- Can include delays/events

```
task load_data;
input [7:0] d;
begin
 data = d;
end
endtask
```

14.45 18. Parameters

Used for reusable configurable modules.

```
parameter WIDTH = 8;
reg [WIDTH-1:0] data;
```

14.45.1 Override at Instantiation

```
fifo #(16) u1 (...);
```

14.46 19. Generate Constructs (1364-2005 Important RTL Feature)

Used for elaboration-time hardware replication.

14.47 Generate For

```
genvar i;
generate
for(i=0;i<8;i=i+1)
 begin
 end
endgenerate
```

14.48 Generate If / Case

Conditional structural generation.

Useful for scalable parameterized RTL.

14.49 20. Timing and Delay Modeling

Delay syntax:

```
#5 a = b;
assign #3 y = a & b;
```

14.49.1 Delay Types

- Rise
- Fall
- Turn-off

Used mainly in simulation/gate-level models.

14.50 21. User Defined Primitives (UDP)

Allows truth-table based primitive creation.

Example uses:

- Custom latch
- Special logic primitive

Less common in modern RTL but present in standard.

14.51 22. Gate-Level Primitive Instances

Built-in primitives:

- and
- or
- nand
- nor
- xor
- buf
- not
- transistor primitives

Example:

```
and g1(y, a, b);
```

14.52 23. Specify Blocks

Used for path delays and timing checks.

```
specify
(a *> y) = 3;
endspecify
```

Used in standard-cell simulation models.

14.53 24. System Tasks and Functions

Common examples:

14.54 Display / Monitor

$display(...)
$monitor(...)
$strobe(...)

14.55 Simulation Control

$finish
$stop
$time

14.56 File I/O

$fopen
$fdisplay
$fclose

14.57 Memory Load

$readmemb
$readmemh

14.58 Waveforms

$dumpfile
$dumpvars

14.59 25. Scheduling Semantics (Very Important)

Verilog simulator uses event queues.

Typical regions:

- Active
- Inactive
- NBA (nonblocking assignment)
- Monitor/Postponed

14.59.1 Why Important

Explains race conditions and why blocking/nonblocking behave differently.

14.60 26. Strength Modeling

Signals may have drive strengths:

- strong
- weak
- pull
- supply

Used for tri-state buses and switch-level models.

Less common in synthesizable RTL.

14.61 27. Compilation Units and Directives

Preprocessor directives:

- `define
- `include
- `ifdef
- `timescale

Example:

```
`timescale 1ns/1ps
```

14.62 28. Synthesizable Subset (Industry Practice)

Although standard allows many features, practical synthesis usually uses:

- Modules
- Parameters
- wire, reg
- assign
- always
- if/case
- loops with static bounds
- tasks/functions (restricted)
- generate blocks

Usually avoided in synthesis:

- delays
- force/release
- specify

- UDPs
- event-based testbench-only code

14.63 29. Common RTL Templates

14.64 Combinational Logic

```
always @(*) begin
  case(sel)
    2'b00: y = a;
    default: y = b;
  endcase
end
```

14.65 Sequential Logic

```
always @(posedge clk or negedge rst_n)
begin
 if(!rst_n) q <= 0;
 else q <= d;
end
```

14.66 30. Common Pitfalls

14.67 Latch Inference

Missing assignment path in combinational block.

14.68 Race Conditions

Multiple blocks updating same signal improperly.

14.69 Mixing Blocking/NBA Incorrectly

Can create simulation mismatch.

14.70 Width Mismatch

Silent truncation or extension.

14.71 Casex Misuse

May mask unknown states.

14.72 31. What 1364-2005 Improved

Compared with earlier versions:

- Better clarification of ambiguous semantics
- Mature generate constructs
- Improved syntax consistency
- Better interoperability with toolchains
- Final cleanup before SystemVerilog transition

14.73 32. Relationship to SystemVerilog

SystemVerilog (IEEE 1800) extends Verilog with:

- logic
- always_comb
- always_ff
- interfaces
- classes
- assertions
- randomization
- packages

But classic Verilog 1364 remains foundational.

14.74 33. Professional Revision Takeaways

- IEEE 1364-2005 is the mature classic Verilog standard.
- Modules are the core design unit.
- Four-state logic (0,1,x,z) is fundamental.
- Use assign for combinational nets.
- Use blocking for combinational procedural logic.
- Use nonblocking for clocked sequential logic.
- Parameters and generate enable scalable RTL.
- Many standard features are simulation-only, not synthesis-friendly.
- Understanding scheduling semantics is critical for debug.
- Most modern RTL still reflects IEEE 1364 coding principles.

14.75 34. Interview Focus Areas

Know these thoroughly:

- Blocking vs nonblocking
- Wire vs reg
- Case vs casex vs casez
- Sensitivity lists
- Latch inference

- FSM coding style
- Generate loops
- Signed vs unsigned behavior
- Race conditions
- Simulation vs synthesis mismatch

15 IEEE 1800-2023 SystemVerilog

16 1. Overview of IEEE 1800-2023

IEEE 1800-2023 is the modern SystemVerilog language standard used for RTL design, verification, assertions, modeling, and hardware/software interface development.

SystemVerilog extends Verilog (IEEE 1364) and unifies:

- RTL design language
- Advanced testbench language
- Object-oriented verification language
- Assertion language
- Functional coverage language
- Interface abstraction language

It is the dominant HDL/HVL standard in ASIC, FPGA, and SoC development.

---

17 2. Why IEEE 1800-2023 Matters

SystemVerilog is used because modern chips are too complex for classic Verilog-only workflows.

It improves:

- Design productivity
- Verification scalability
- Reusability
- Readability
- Strong typing
- Testbench automation
- Assertion-driven verification
- Coverage-driven closure

Most professional semiconductor flows today use SystemVerilog for both RTL and verification.

---

18 3. Evolution of the Standard

Standard	Importance
Verilog 1364-2005	Final standalone Verilog standard
IEEE 1800-2005	Initial SystemVerilog integration
IEEE 1800-2009	Major maturity improvements
IEEE 1800-2012	Widely adopted verification features
IEEE 1800-2017	Long-term production standard
IEEE 1800-2023	Current refined modern standard

IEEE 1800-2023 mainly consolidates language clarity, consistency, corrections, and continued modernization.

---

19 4. Main Language Domains

SystemVerilog contains five major usage domains:

Domain	Purpose
RTL Design	Synthesizable hardware design
Verification	Testbench, stimulus, checking
Assertions	Formal and simulation properties
Coverage	Measure verification completeness
Modeling	High-level and mixed abstraction models

---

20 5. Backward Compatibility with Verilog

SystemVerilog supports almost all Verilog RTL code.

Typical legacy code using:

- module
- assign
- always
- wire
- reg

can usually compile directly.

SystemVerilog adds improved replacements for many old constructs.

---

21 6. Improved Data Types

22 6.1 `logic`

Most commonly used replacement for wire/reg confusion.

```
logic a;
logic [7:0] data;
```

22.0.1 Why Important

Classic Verilog required deciding between wire and reg. SystemVerilog logic simplifies signal declaration.

Use logic for most single-driver RTL signals.

---

23 6.2 Two-State Types

Use when x/z states are unnecessary.

- bit
- byte
- shortint
- int
- longint

Example:

```
bit enable;
int count;
```

Useful for testbench speed and software-like variables.

---

24 6.3 Four-State Types

Preserve hardware semantics:

- logic
- integer
- packed vectors

Use for RTL signals.

---

25 6.4 Signed Types

```
logic signed [15:0] a;
int signed x;
```

Explicit signed arithmetic reduces ambiguity.

---

26 7. Packed and Unpacked Arrays

27 7.1 Packed Arrays

Bit-level vectors.

```
logic [7:0] data;
```

28 7.2 Unpacked Arrays

Collections of elements.

```
logic [7:0] mem [0:255];
```

256 bytes.

29 7.3 Multi-Dimensional Arrays

```
logic [7:0] img [0:479][0:639];
```

Useful for memories, matrices, image buffers.

---

30 8. Structures and Unions

31 8.1 Struct

Groups related fields.

```
typedef struct packed {
  logic valid;
  logic [7:0] addr;
  logic [31:0] data;
} pkt_t;
```

Useful for buses and transactions.

32 8.2 Union

Multiple interpretations of same storage.

Used carefully in protocols and packed data overlays.

---

33 9. Enumerations

Strongly typed symbolic states.

```
typedef enum logic [1:0] {
  IDLE, RUN, DONE, ERR
} state_t;
```

Benefits:

- Better readability
- Safer FSM coding
- Easier waveform debug

---

34 10. Typedef

Creates reusable types.

```
typedef logic [31:0] word_t;
```

Encouraged for scalable design.

---

35 11. Procedural Blocks – Safer Replacements

36 11.1 `always_comb`

For combinational logic.

```
always_comb begin
  y = sel ? a : b;
end
```

Benefits:

- Automatic sensitivity
- Tool checks for combinational intent

---

37 11.2 always_ff

For sequential logic.

```
always_ff @(posedge clk)
  q <= d;
```

Ensures flip-flop intent.

---

38 11.3 always_latch

For intentional latch logic.

```
always_latch
 if(en) q <= d;
```

---

39 12. Procedural Assignment Rules

39.1 Blocking =

Used mainly in combinational calculations.

39.2 Nonblocking <=

Used in sequential clocked logic.

Same principle as Verilog, but SystemVerilog tools enforce intent better with always_ff.

---

40 13. Interfaces

One of the most important SystemVerilog features.

Interfaces bundle related signals and protocols.

```
interface bus_if;
 logic clk;
 logic req;
 logic gnt;
 logic [31:0] addr;
endinterface
```

40.0.1 Benefits

- Cleaner module connections
- Reusable bus definitions

- Easier protocol management

---

41 14. Modports

Restrict interface access directions for modules.

```
modport master (output req, input gnt);
modport slave  (input req, output gnt);
```

Improves correctness.

---

42 15. Packages

Used for shared declarations.

```
package common_pkg;
 typedef logic [31:0] word_t;
 parameter DEPTH = 16;
endpackage
```

Imported with:

```
import common_pkg::*;
```

Used heavily in professional codebases.

---

43 16. Functions and Tasks

Enhanced over Verilog.

- Typed arguments
- Default arguments
- Automatic recursion
- Void functions
- Better scoping

Example:

```
function automatic int add(int a,b);
  return a+b;
endfunction
```

---

44 17. Parameterization

Supports scalable reusable modules.

```
module fifo #(parameter DEPTH=16, WIDTH=8);
```

Used with generate constructs.

---

45 18. Generate Blocks

Compile-time structural generation.

```
for(genvar i=0;i<8;i++) begin
end
```

Used for:

- Replication
- Width scaling
- Optional hardware

---

46 19. Object-Oriented Programming (Verification)

SystemVerilog adds classes for testbench development.

```
class packet;
 rand bit [7:0] addr;
 rand bit [31:0] data;
endclass
```

Used in UVM and reusable verification environments.

---

47 20. Core OOP Features

- Class
- Object handles
- Inheritance
- Polymorphism
- Encapsulation
- Virtual methods

Essential for enterprise-scale verification.

---

48 21. Randomization

Constraint-driven random stimulus generation.

```
class pkt;
 rand bit [7:0] addr;
 constraint c { addr < 100; }
endclass
```

Used to explore corner cases automatically.

---

49 22. Assertions (SVA)

SystemVerilog Assertions verify protocol and timing behavior.

Example:

```
assert property (@(posedge clk) req |-> ##1 gnt);
```

Meaning: If req occurs, gnt must occur next cycle.

Used in:

- Simulation
- Formal verification
- Emulation

---

50 23. Immediate vs Concurrent Assertions

50.1 Immediate Assertions

Procedural checks now.

```
assert(a == b);
```

50.2 Concurrent Assertions

Temporal clock-based properties.

Used for protocols.

---

51 24. Coverage

Measures verification completeness.

51.1 Functional Coverage

Tracks scenarios hit.

```
covergroup cg;
 coverpoint opcode;
endgroup
```

51.2 Code Coverage

Tool measures:

- line
- branch
- toggle
- FSM

---

52 25. Constrained Random Verification

Professional methodology combining:

- random transactions
- assertions
- coverage
- scoreboards

This is core modern verification flow.

---

53 26. Mailboxes, Semaphores, Events

Used in testbench synchronization.

53.1 Mailbox

Producer-consumer communication.

53.2 Semaphore

Shared resource control.

53.3 Event

Process synchronization.

---

54 27. Processes and Concurrency

Supports:

- fork ... join
- process control
- threads
- wait semantics

Useful in testbenches.

---

55 28. DPI (Direct Programming Interface)

Connects C/C++ and SystemVerilog.

Used for:

- golden models
- algorithm acceleration
- software integration
- legacy model reuse

---

56 29. Clocking Blocks

Helps synchronize testbench interaction.

```
clocking cb @(posedge clk);
 input data;
 output req;
endclocking
```

Reduces race conditions.

---

57 30. Program Blocks

Introduced for testbench ordering semantics.

Less common today because classes/UVM dominate.

---

58 31. Virtual Interfaces

Allows classes to access interfaces.

Critical for UVM drivers/monitors.

```
virtual bus_if vif;
```

---

59 32. Streaming Operators

Used for packing/unpacking bit streams.

```
{<<8{data}}
```

Useful for protocol serialization.

---

60 33. Queues

Dynamic ordered collections.

```
int q[$];
q.push_back(5);
```

---

61 34. Dynamic Arrays

Resizable arrays.

```
int arr[];
arr = new[10];
```

---

62 35. Associative Arrays

Indexed by arbitrary keys.

```
int mem[string];
```

Useful in scoreboards.

---

63 36. Strings

Native string support.

```
string name = "pkt0";
```

Better than legacy packed vectors.

---

64 37. Casting

Strong type conversion.

```
state_t'(2)
int'(x)
```

Useful with enums and classes.

---

65 38. Scheduler and Simulation Regions

SystemVerilog refines event scheduling with regions for assertions and testbench ordering.

Important for avoiding races between:

- DUT RTL
- Assertions
- Testbench stimulus

---

66 39. Synthesizable Subset

Common RTL synthesizable constructs:

- modules
- interfaces (tool dependent / common now)
- logic
- always_comb
- always_ff
- enums
- structs
- parameters
- generate

Usually non-synthesizable:

- classes
- randomization
- mailboxes
- covergroups
- assertions (mostly verification intent)
- dynamic testbench constructs

---

67 40. FSM Best Practice Example

```
typedef enum logic [1:0] {IDLE,RUN,DONE} state_t;

state_t state,next;

always_ff @(posedge clk)
 state <= next;

always_comb begin
 next = state;
 case(state)
   IDLE: if(start) next = RUN;
   RUN : if(done)  next = DONE;
 endcase
end
```

This is cleaner and safer than classic Verilog FSM style.

---

68 41. UVM Relationship

UVM (Universal Verification Methodology) is built on SystemVerilog classes and features.

Uses:

- factory
- sequences
- agents
- drivers
- monitors
- scoreboards

SystemVerilog knowledge is required before UVM mastery.

---

69 42. IEEE 1800-2023 Focus Areas

Compared with older versions, 2023 emphasizes:

- Clarified semantics
- Better consistency
- Updated examples and wording
- Mature assertion/testbench behavior
- Continued industry alignment

---

70 43. Common Professional Pitfalls

70.1 RTL Pitfalls

- Mixing blocking/nonblocking incorrectly
- Missing defaults in combinational logic
- Width/sign mismatches
- Misusing interfaces

70.2 Verification Pitfalls

- Weak constraints
- Poor coverage planning
- Over-randomization without checking
- Race conditions without clocking blocks

---

71 44. Interview Focus Areas

Know these thoroughly:

- logic vs wire/reg
- always_comb, always_ff
- packed vs unpacked arrays
- enum FSM coding
- interface/modport
- classes and randomization
- assertions
- covergroups
- mailbox vs queue
- DPI
- virtual interface

---

72 45. Professional Revision Takeaways

- IEEE 1800-2023 is the modern standard for hardware design and verification.
- It extends Verilog with stronger RTL constructs and enterprise-grade verification features.
- Use logic, always_comb, and always_ff for modern RTL.
- Use enums, structs, packages, and interfaces for scalable architecture.
- Use classes, randomization, assertions, and coverage for verification.
- SystemVerilog is the foundation of modern ASIC/SoC verification flows.
- Strong SystemVerilog knowledge is essential for FPGA, ASIC, DV, and UVM careers.

73 Hardware Description Language

---

74 Table of Contents

---

75 Verilog vs VHDL

VHDL	Verilog
Strongly typed	Weakly typed
Easier to understand	Less code to write
More natural in use	More of a hardware modeling language
Wordy	Concise
Non-C-like syntax	Similarities to the C language
Variables must be described by data type	A lower level of programming constructs
Widely used for FPGAs and military	A better grasp on hardware modeling
More difficult to learn	Simpler to learn
Complex data tyepes	Simpler data tyepes

---

76 Verilog vs SystemVerilog

System Verilog and verilog both Both are IEEE standards, Verilog is IEEE 1364 -2005 (Latest Version)and System verilog is IEEE 1800 - 2017(Latest version). Verilog is a Hardware Description Language , whereas System verilog is a combination of Hardware Description Language (HDL) and Hardware Verification Language (HVL). So, System verilog can be considered as an extension or a superset of Verilog.

- **History** Verilog, is the popular Hardware Description Language invented in early 1980's. Whereas, System Verilog started initially with a name, as Superlog in 2002 and was standardized in 2005 with its own name. System verilog for RTL design is an extension of verilog (2005) and has all of its features.System verilog for verification uses Object-oriented programming techniques.
- **Data types** Verilog has majorly two datatypes – Reg and Wire which are 4 valued logic 0,1,x,z. Whereas, System verilog has logic(inclusive of Wire & Reg), int, shortint, longint, logic, bit, real, realtime, reg, chandle, user defined data type, etc. which are both combination of 4 and 2 valued logic.

- **Memory** Verilog does not allow packed array concept and the lifetime of memories will be static. Whereas, System verilog allows packed array declaration and the lifetime of memories can be dynamic.
- **Procedural Block** Verilog has a general purpose always block to model different types of hardware structures. Whereas, System verilog uses three different procedural blocks namely, always_comb , always_ff and always_latch intended to model specific type of hardware description.
- **Construction** Verilog design is based on hierarchy of modules, where modules encapsulate the design hierarchy, and communicate with other modules through ports. Whereas , System verilog uses Class based design.
- **Interface** Verilog ports (larger designs) for describing a module's connectivity with other module is difficult. Whereas, System verilog uses interface to reduce the redundancy of port declarations between modules.
- **Random** Verilog uses inbuit system functions like $random and $urandom, whereas System Verilog uses a method called Randomize().
- **Constraints** Verilog does not support any control over the variable to be randomized, whereas System verilog uses constraints to have a control of what is being randomized.
- **TB Environment** Verilog does not support for having reusable testbenches and so for complex designs verification will be a milestone whereas, System verilog supports for having reusable testbenches.
- **Synchronisation** SystemVerilog uses interface construct which has used for bunching of all the signals along with clocking block which is used for synchronisation unlike Verilog in which instantiation with the DUT becomes tedious because of large number of signals.

---

77 VHDL

78 Introduction

VHDL is a Department of Defense–mandated hardware description language. (The V in VHDL stands for the first letter in VHSIC, an acronym for very high-speed integrated circuit.) - VHDL can model the behavior and structure of digital system at multiple abstraction levels. - VHDL is strongly typed and very deterministic.